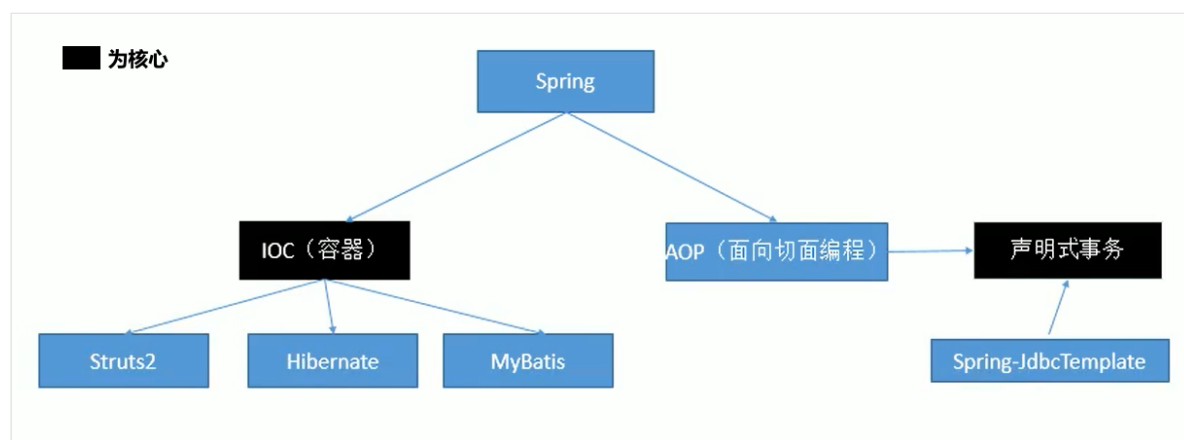


Spring(IOC+AOP雷神13-?集)

1. IOC和DI



1.1. IOC: (Inversion Of Control)

控制: 资源的获取方式;

主动式: (要什么资源都自己创建即可)

```
1 BookServlet{
2     BookService bs = new BookService();
3     AirPlane ap = newAirPlane(); //复杂对象的创建就是比较庞大的工程;
4 }
```

被动式: 资源的获取不是我们自己创建, 而是交给一个容器来创建和设置;

```
1 BookServlet{
2     BookService bs;
3     public void test01(){
4         bs.checkout(); //没有创建bs对象就使用的话会空指针异常, 我们这里交给容器来创建
5     }
6 }
```

容器: 管理所有的组件(有功能的类); 假设, BookServlet受容器管理, BookService也受容器管理; 容器可以自动的探查出那些组件(类)需要用到另一些组件(类); 容器帮我们创建BookService对象, 并把BookService对象赋值过去;

容器: 主动的new资源变为被动的接受资源;

1.2. DI:(Dependency Injection) 依赖注入;

容器能知道哪个组件(类) 运行的时候, 需要另外一个类(组件); 容器通过反射的形式, 将容器中准备好的BookService对象注入(利用反射给属性赋值) 到BookServlet中;

只要是容器管理的组件, 都能使用容器提供的强大功能;

1.3. 实验1:通过IOC容器创建对象，并为属性赋值(HelloWorld)★

(通过各种方式给容器中注册对象(婚介所注册会员))

以前是自己new对象, 现在所有的对象交给容器创建; 给容器中注册组件;

以后框架编写流程;

HelloWorld:

1.3.1. 导包

```
spring-beans-4.0.0.RELEASE.jar
spring-context-4.0.0.RELEASE.jar
spring-core-4.0.0.RELEASE.jar
spring-expression-4.0.0.RELEASE.jar
commons-logging-1.1.3.jar
Spring运行的时候依赖一个日志包; 没有就报错;
```

1.3.2. 配置

spring的配置文件中, 集合了spring的ioc容器管理的所有组件(会员清单)

创建一个Spring Bean Configuration File(Spring的bean配置文件, 没有的话去marketplace下载Spring Tools3 Add-On)

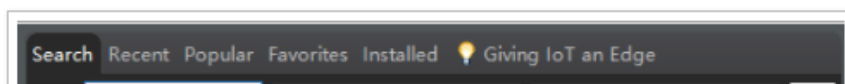
eclipse 新建文件没有 Spring Bean Configuration...

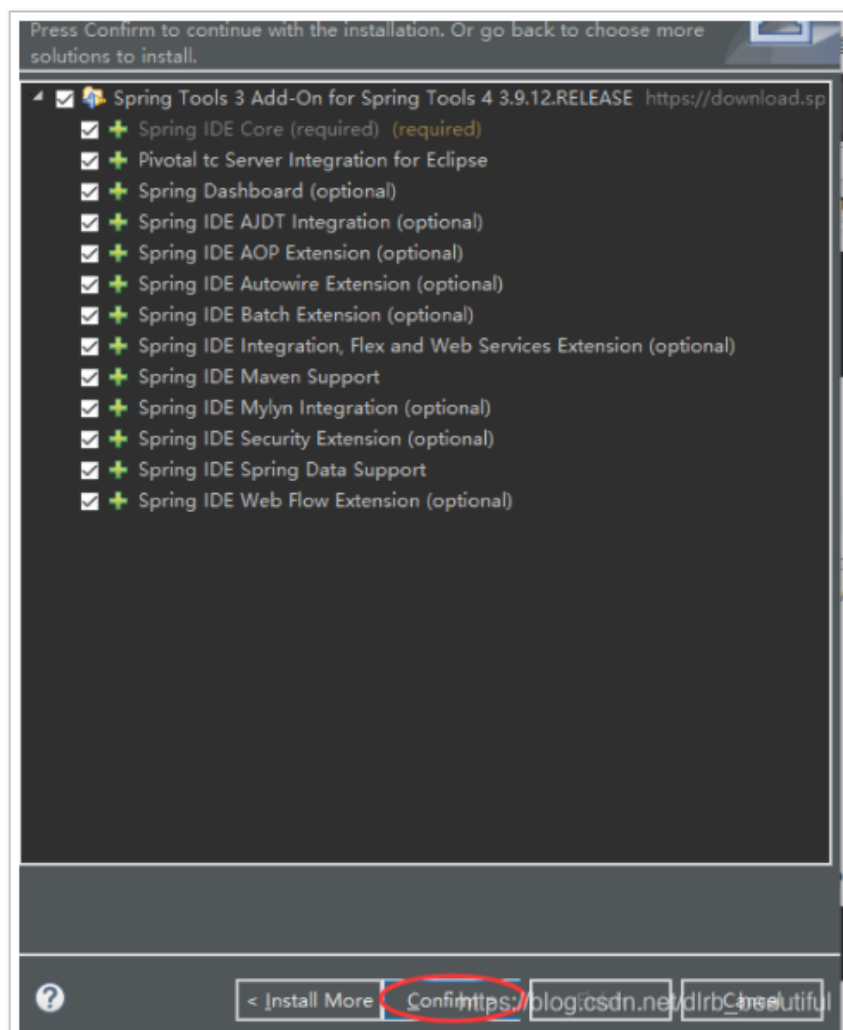
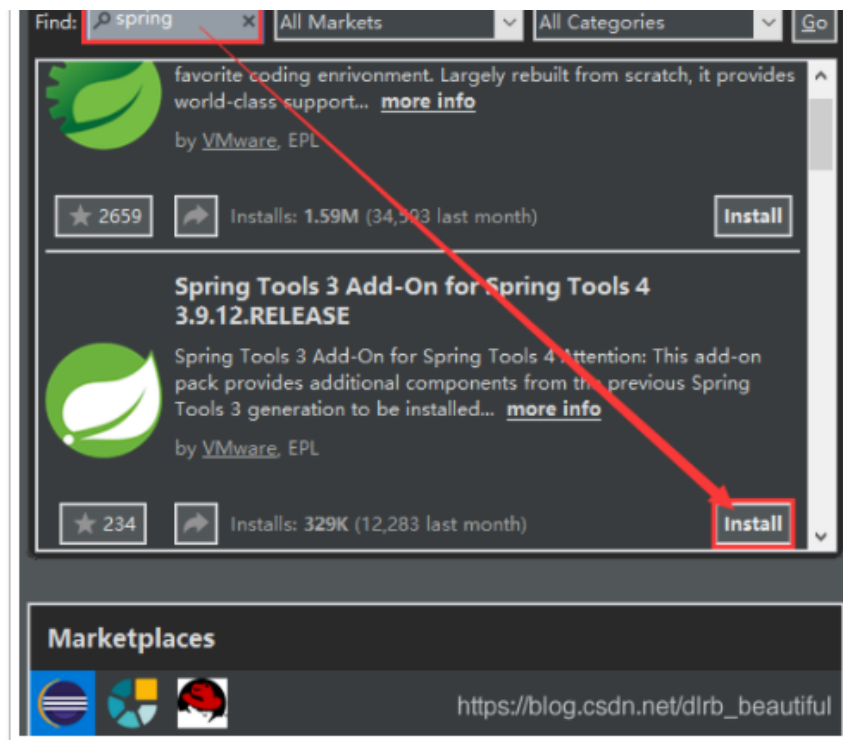
新建 spring 配置文件的时候没有找到
Spring Bean Configuration File 文件
解决办法:

下载 spring 插件:

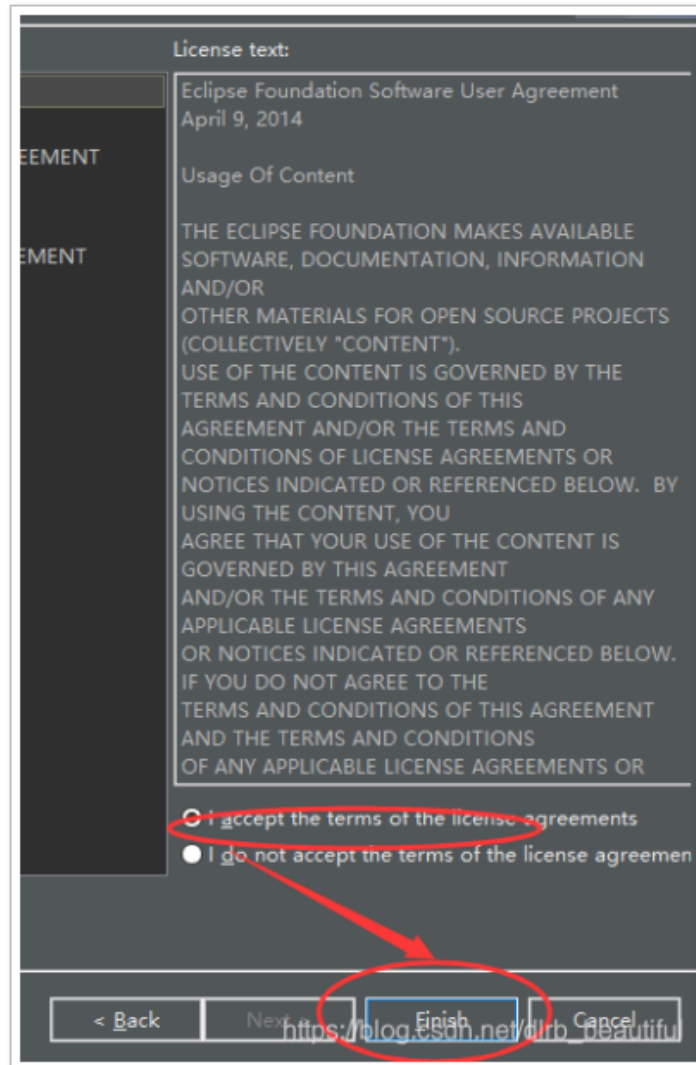
点击 help->eclipse marketplace

搜索 spring: install





选择 confirm



选择 accept, finish

接下来会下载一段时间,

下载完成后会提示重启

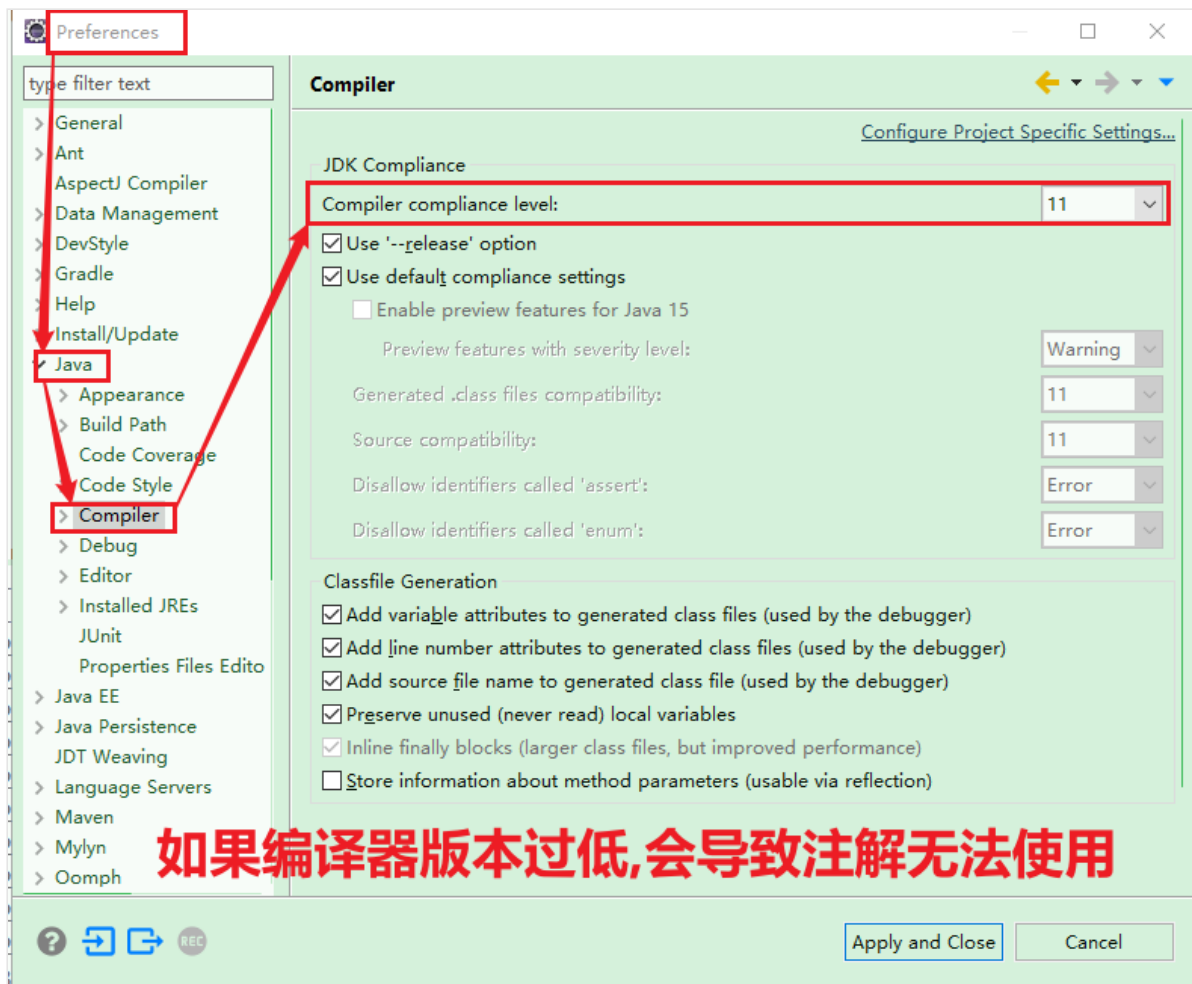
重新启动一下 eclipse 就完成了。

全文完

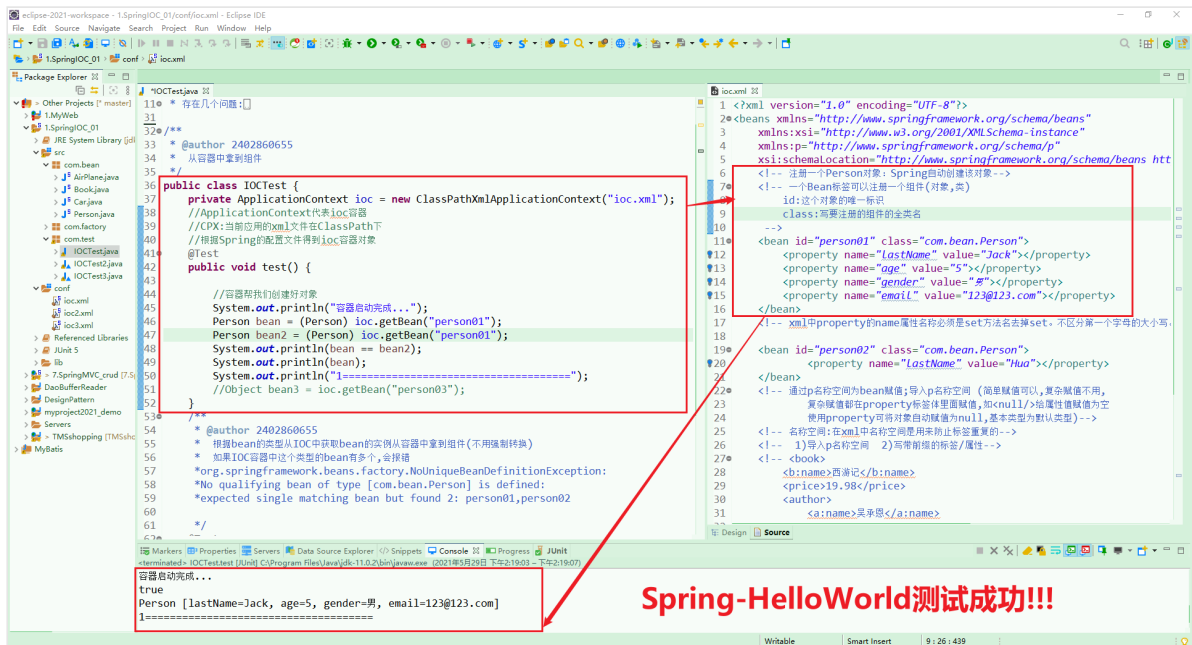
本文由 简悦 SimpRead 优化, 用以提升阅读体验

使用了 全新的简悦词法分析引擎 beta, 点击查看详细说明





1.3.3. 测试



1.3.4. 一些问题和细节

1.3.4.1. 类路径是什么?

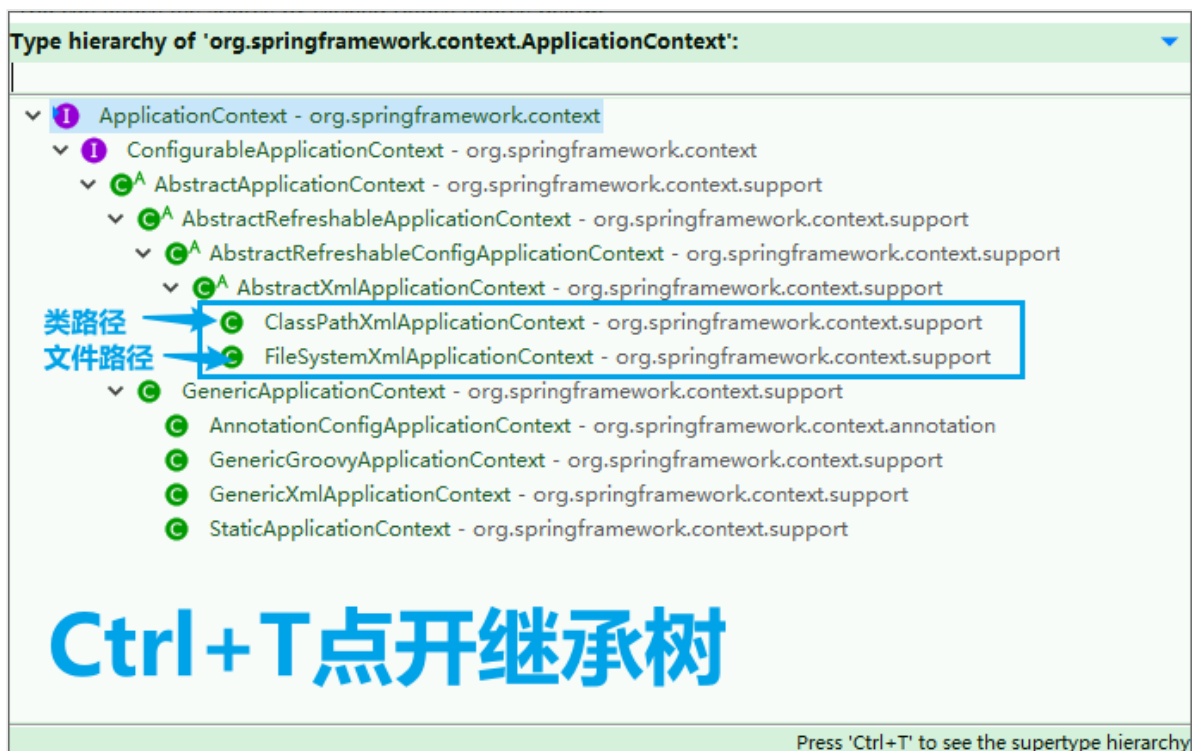
java项目: src为源码包,源码包会进入bin目录下,bin目录为类路径:ClassPath的开始

web项目 /WEB-INF/classes/



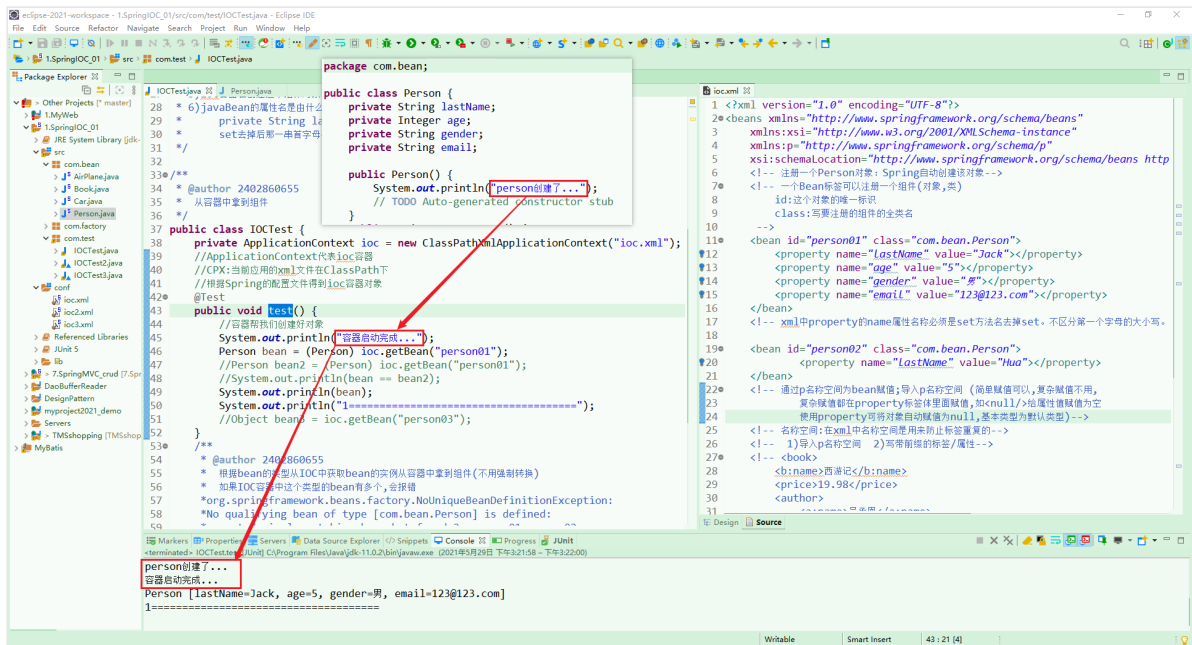
1.3.4.4. ApplicationContext是什么？

ApplicationContext (IOC容器的接口), 可以以两种方式获取xml文件



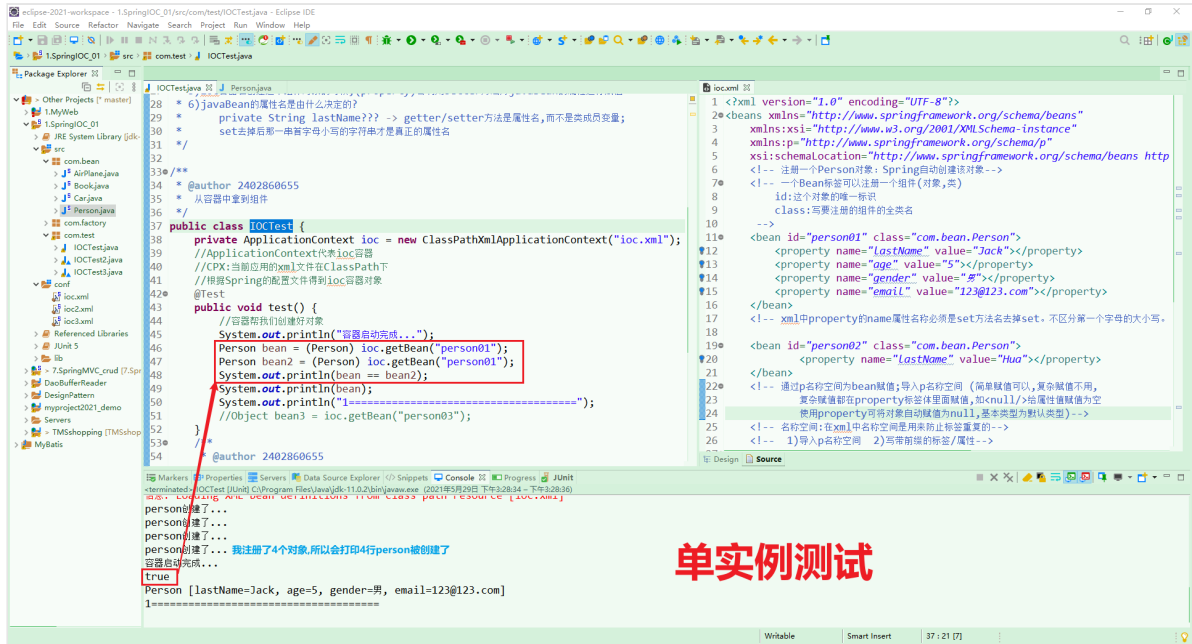
1.3.4.5. 对象由容器创建,她是什么时候被创建好的？

容器中对象的创建,在容器创建完成的时候就已经创建好了



1.3.4.6. 同一个组件(或者说对象)在ioc容器中是单实例的吗?

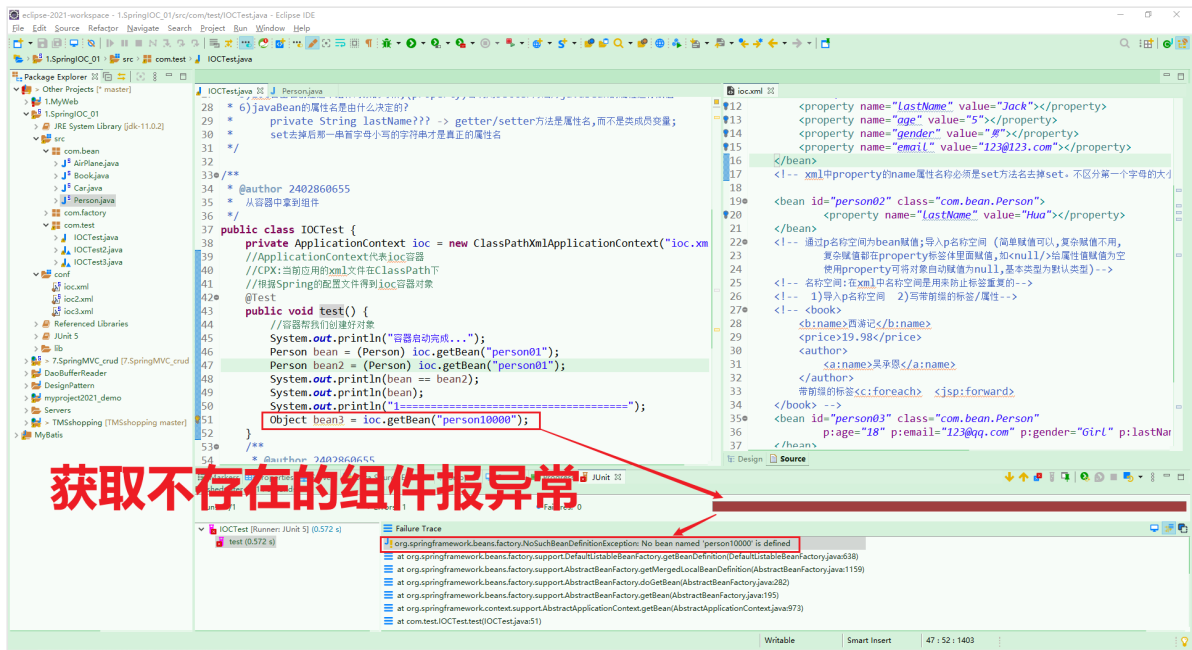
是单实例的



1.3.4.7. 获取未注册的组件,将会报异常

org.springframework.beans.factory.NoSuchBeanDefinitionException:

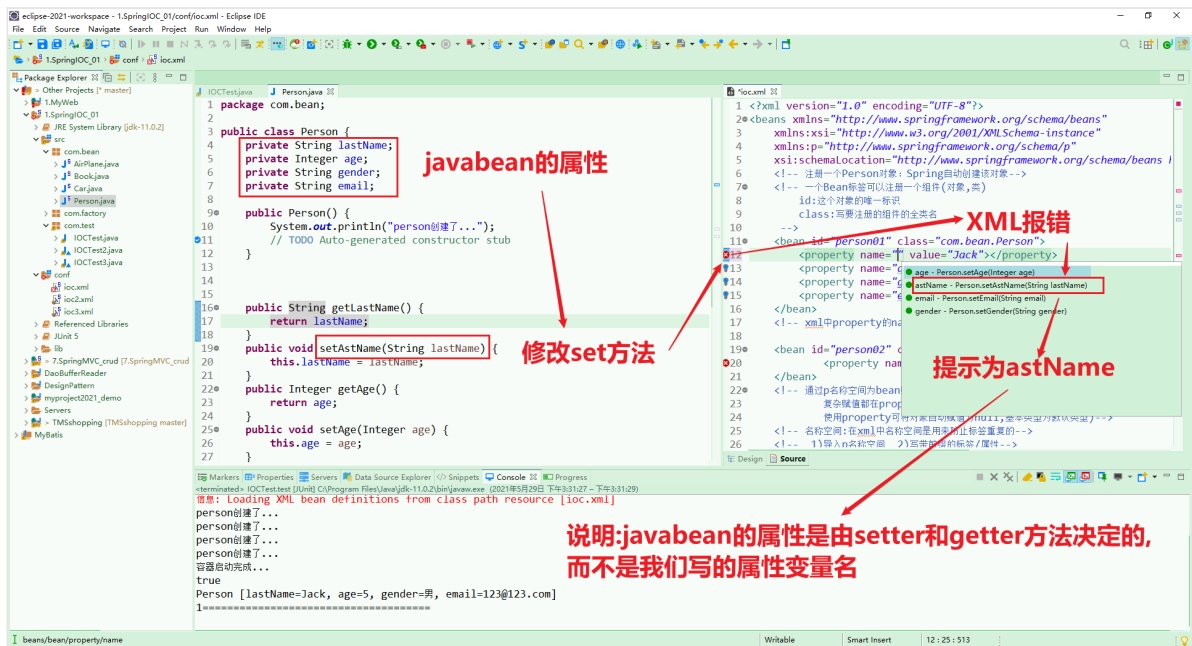
No bean named 'person10000' is defined



1.3.4.8. JavaBean的属性名是由什么决定的？

JavaBean的属性名, 是set方法去掉后那一串首字母小写的字符串才是真正的属性名

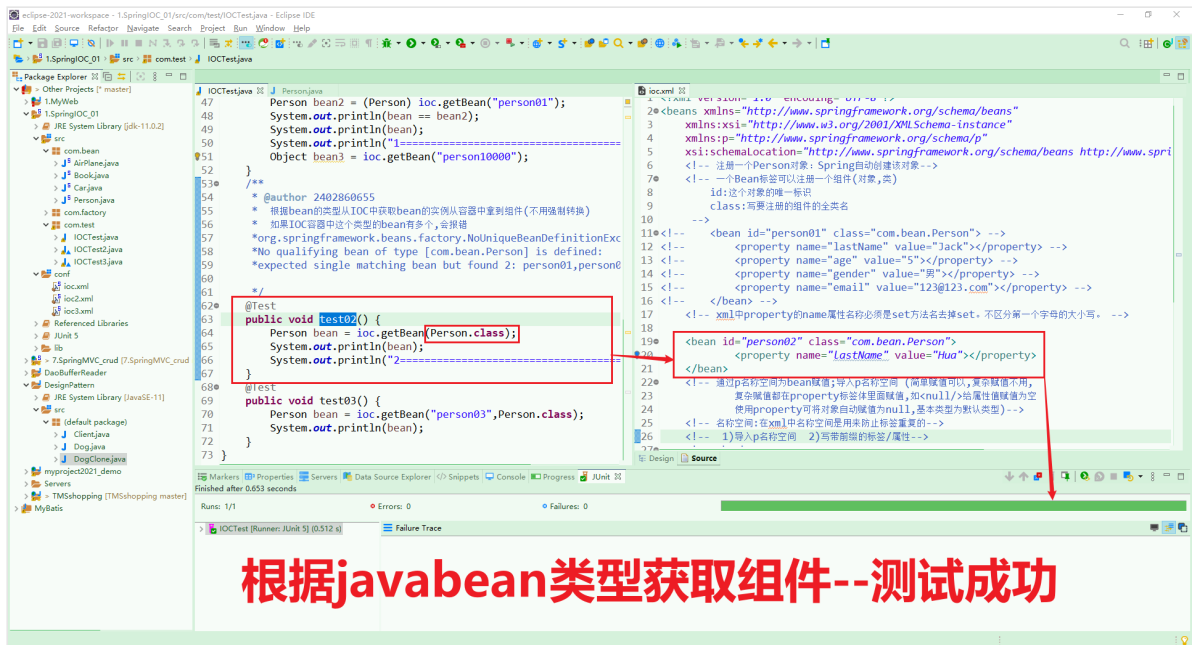
而不是类成员变量: private String lastName



1.4. 实验2:根据bean的类型, 从IOC中获取bean的实例(从容器中拿到组件)★(雷神17集)

1.4.1. 符合javabean类型的组件只有一个:

根据bean的类型从IOC中获取bean的实例从容器中拿到组件(不用强制转换)



根据javabean类型获取组件--测试成功

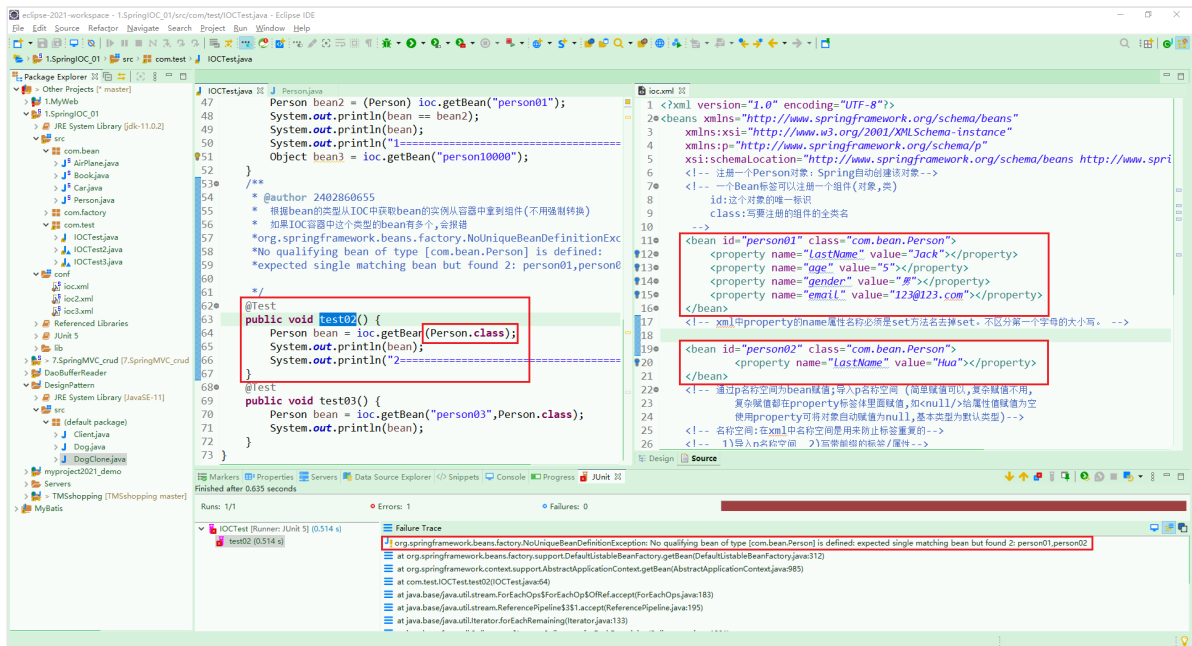
1.4.2. 符合javabean类型的组件有多个:

如果IOC容器中这个类型的bean有多个,会报错

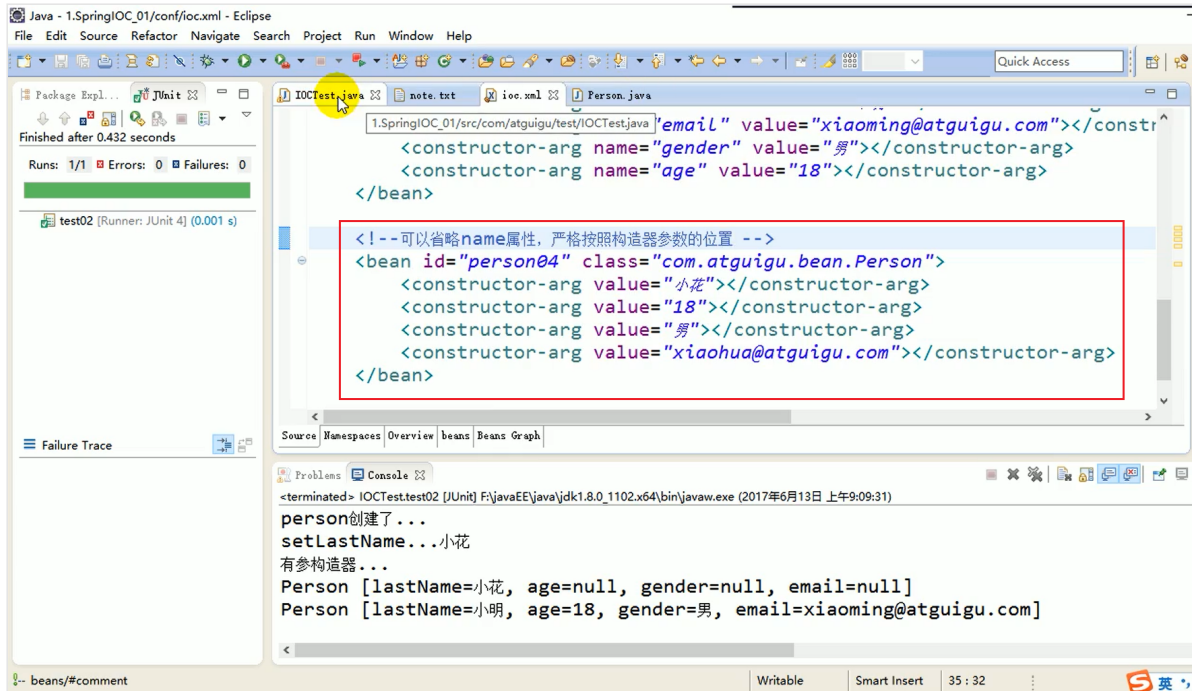
org.springframework.beans.factory.NoUniqueBeanDefinitionException:

No qualifying bean of type [com.bean.Person] is defined:

expected single matching bean but found 2: person01,person02



1.5.1.2. 可以省略name属性,严格按照构造器参数的位置赋值(不建议使用,雷神19集)



1.5.1.3. 重载的构造器, 赋值不确定,使用type(不建议使用,雷神19集)


```

public Person(String lastName, Integer age, String gender){
    this.lastName = lastName;
    this.age = age;
    this.gender = gender;
    System.out.println("三个参数的构造器...age");
}

public Person(String lastName, String email, String gender){
    this.lastName = lastName;
    this.email = email;
    this.gender = gender;
    System.out.println("三个参数的构造器...email");
}

```

```

<!--public Person(String lastName, Integer age, String gender) -->
<!--public Person(String lastName, String email, String gender) -->
<bean id="person05" class="com.atguigu.bean.Person">
    <constructor-arg value="小丽"></constructor-arg>
    <constructor-arg value="10"></constructor-arg>
    <constructor-arg value="男"></constructor-arg>
</bean>
</beans>

```

**如果有重载的构造器,
赋值有一定随机性**

Source Namespaces Overview beans Beans Graph

Problems Console

<terminated> IOCTest.test02 [JUnit] F:\javaEE\java\jdk1.8.0_1102.x64\bin\javaw.exe (2017年6月13日 上午9:17:06)

有参构造器...

三个参数的构造器...email

Person [lastName=小花, age=null, gender=null, email=null]

Person [lastName=小丽, age=null, gender=男, email=10]

```

<!--public Person(String lastName, Integer age, String gender) -->
<!--public Person(String lastName, String email, String gender) -->
<!-- 重载的情况下type可以指定参数的类型 -->
<bean id="person05" class="com.atguigu.bean.Person">
    <constructor-arg value="小丽"></constructor-arg>
    <constructor-arg value="10" index="1" type="java.lang.Integer"></constructor-arg>
    <constructor-arg value="男"></constructor-arg>
</bean>
</beans>

```

**重载情况下用type指定参数的类型
(此处index="1",没有实际区分作用)**

Source Namespaces Overview beans Beans Graph

Problems Console

<terminated> IOCTest.test02 [JUnit] F:\javaEE\java\jdk1.8.0_1102.x64\bin\javaw.exe (2017年6月13日 上午9:18:01)

有参构造器...

三个参数的构造器...age

Person [lastName=小花, age=null, gender=null, email=null]

Person [lastName=小丽, age=10, gender=男, email=null]

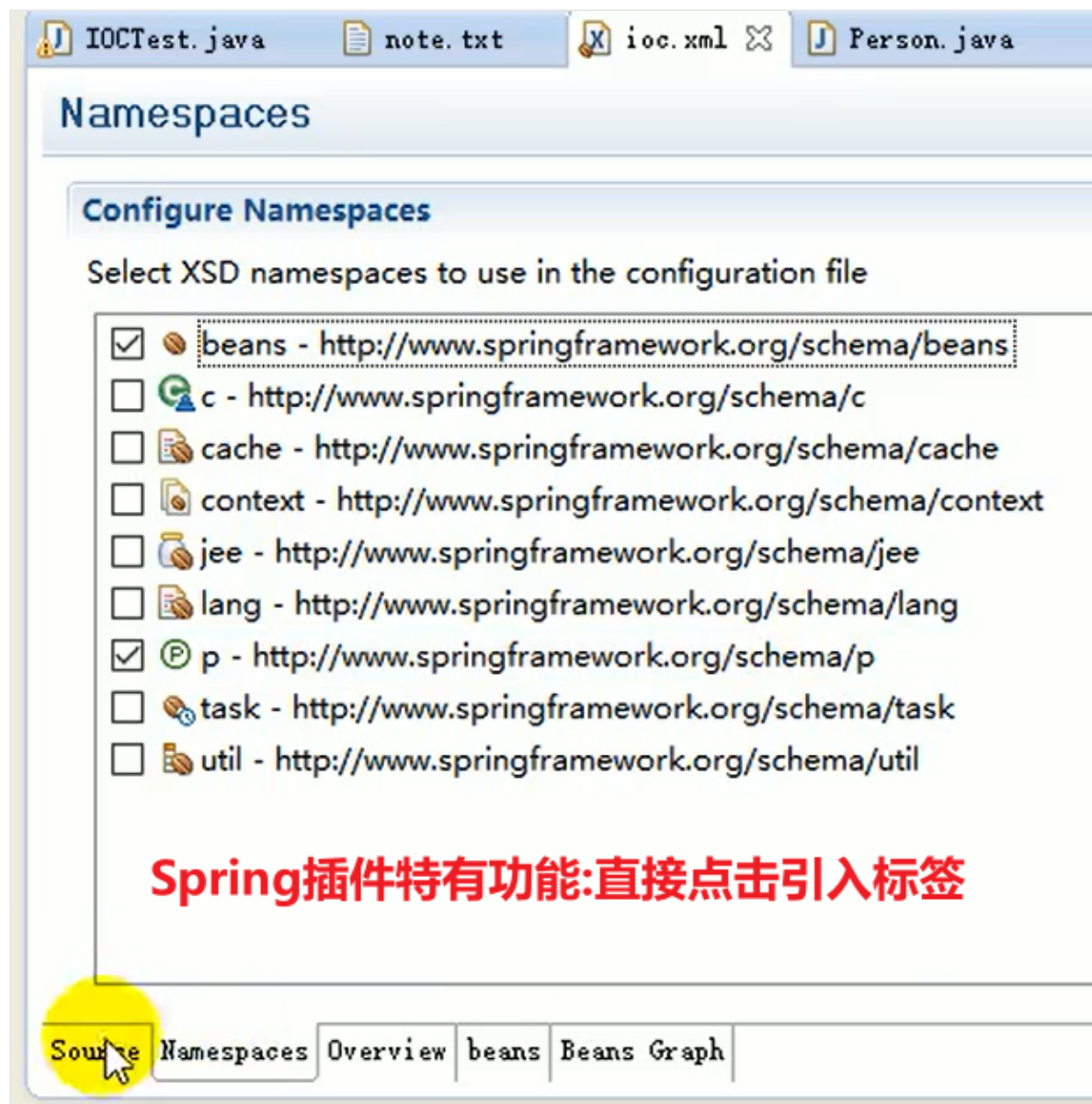
1.5.1.4. 通过p名称空间为bean赋值(雷神20集)

1.5.1.4.1. 名称空间: 用前缀来区分变量的范围

```

1 <book>
2   <b:name>西游记</b:name>
3   <price>19.98</price>
4   <author>
5     <a:name>吴承恩</a:name>
6   </author>
7   带前缀的标签<c:foreach> <jsp:forward>
8 </book>

```

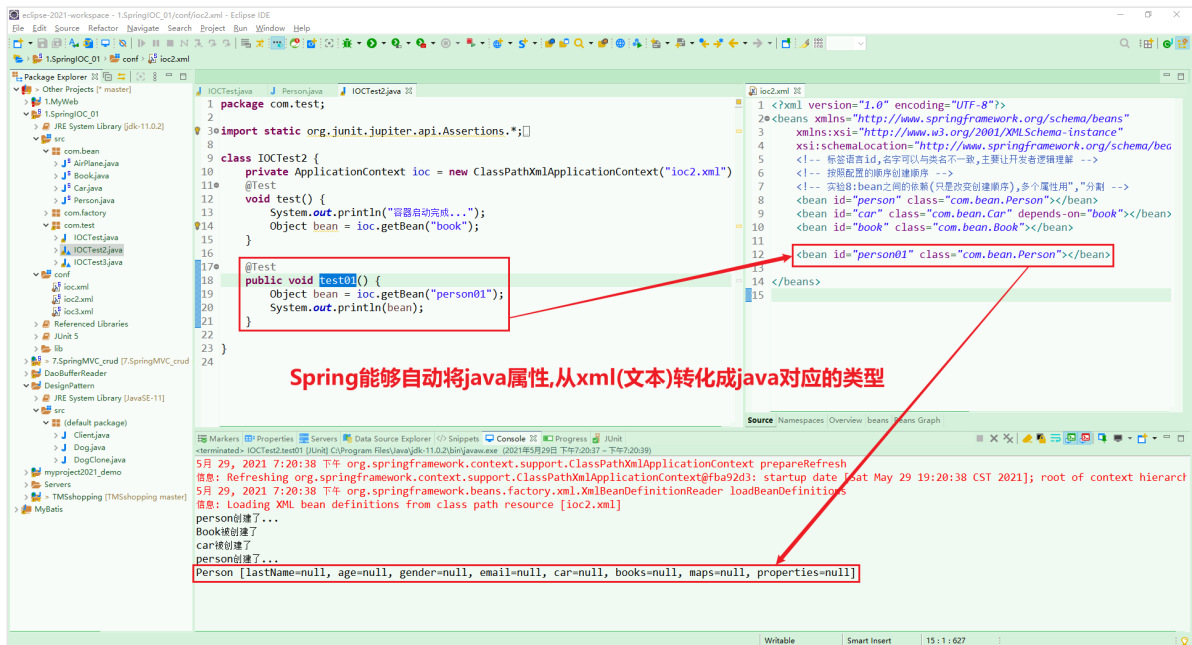


```
<bean id="person06" class="com.atguigu.bean.Person"
      p:age="18" p:email="xiaoming@atguigu.com" p:lastName="哈哈" p:gender
</bean>
```

关于p标签赋值:

简单赋值可以,复杂赋值不用,
复杂赋值都在property标签体里面赋值,如给属性值赋值为空
使用property可将对象自动赋值为null,基本类型为默认类型

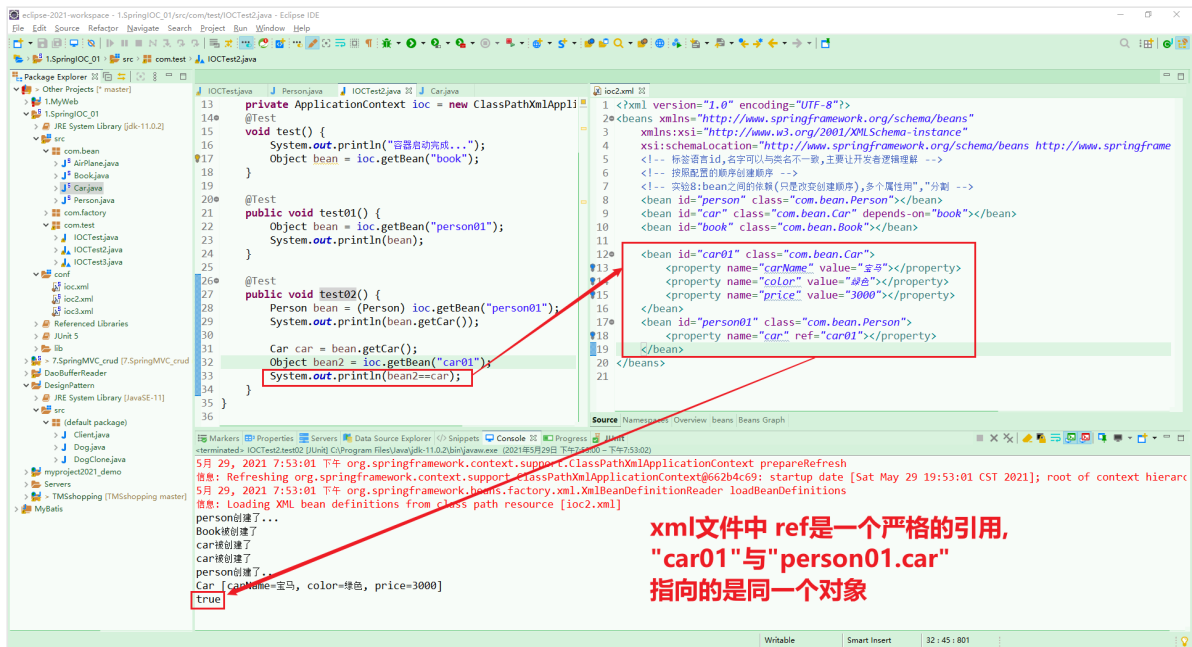
1.6. 实验4:正确为各种属性赋值



1.6.1. 测试使用null值

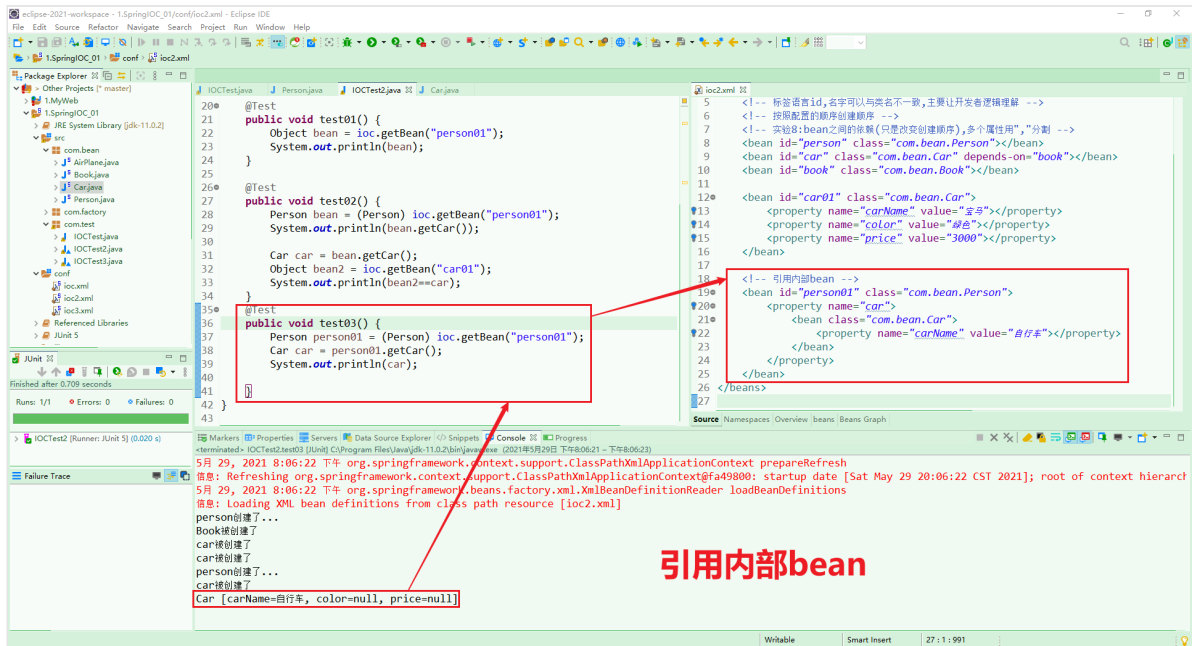


1.6.2. ref引用外部的值

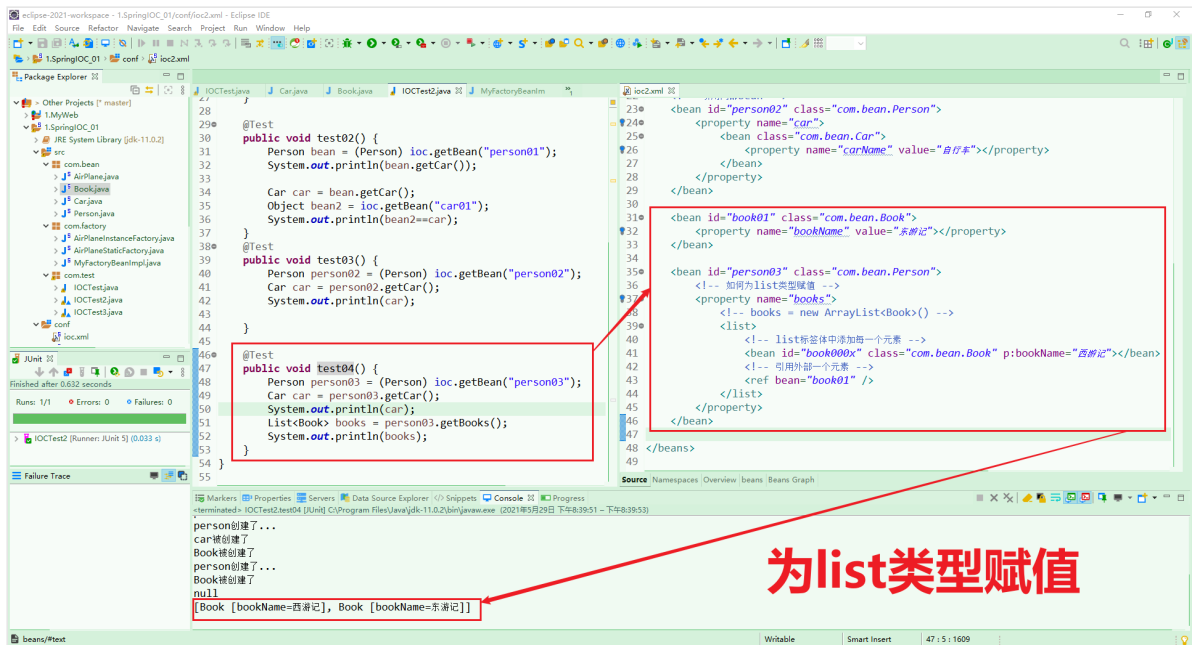


1.6.3. 引用内部bean

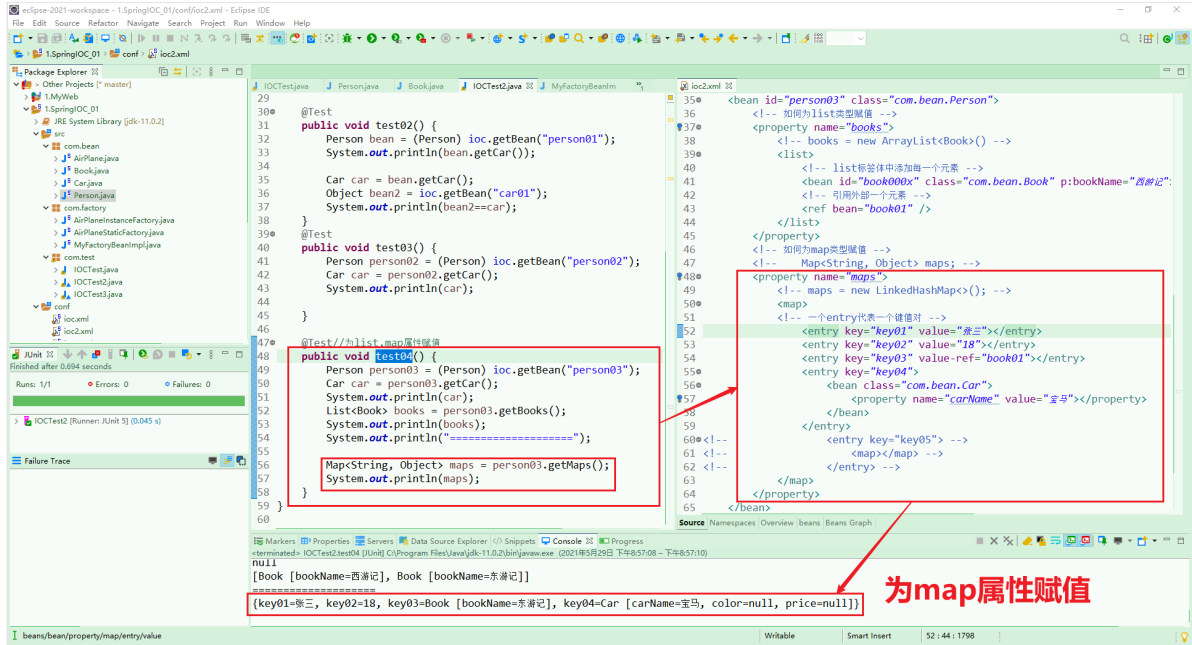
内部bean id写不写都一样,不能通过ioc容器,直接取



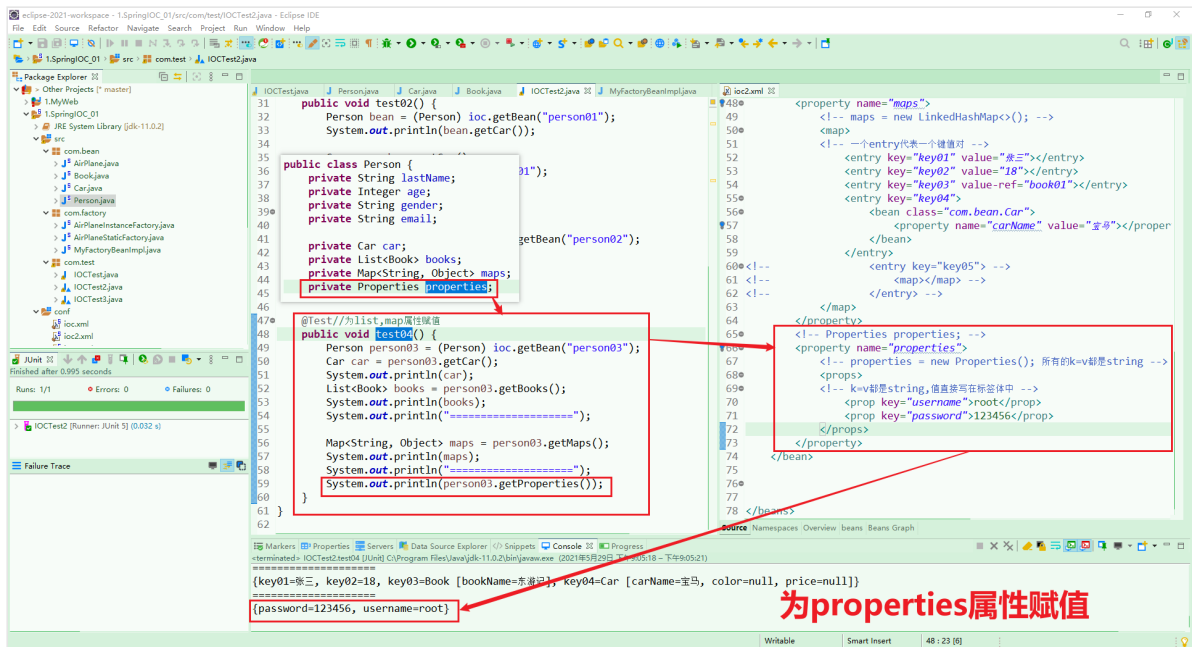
1.6.4. 为list属性赋值(雷神24集)



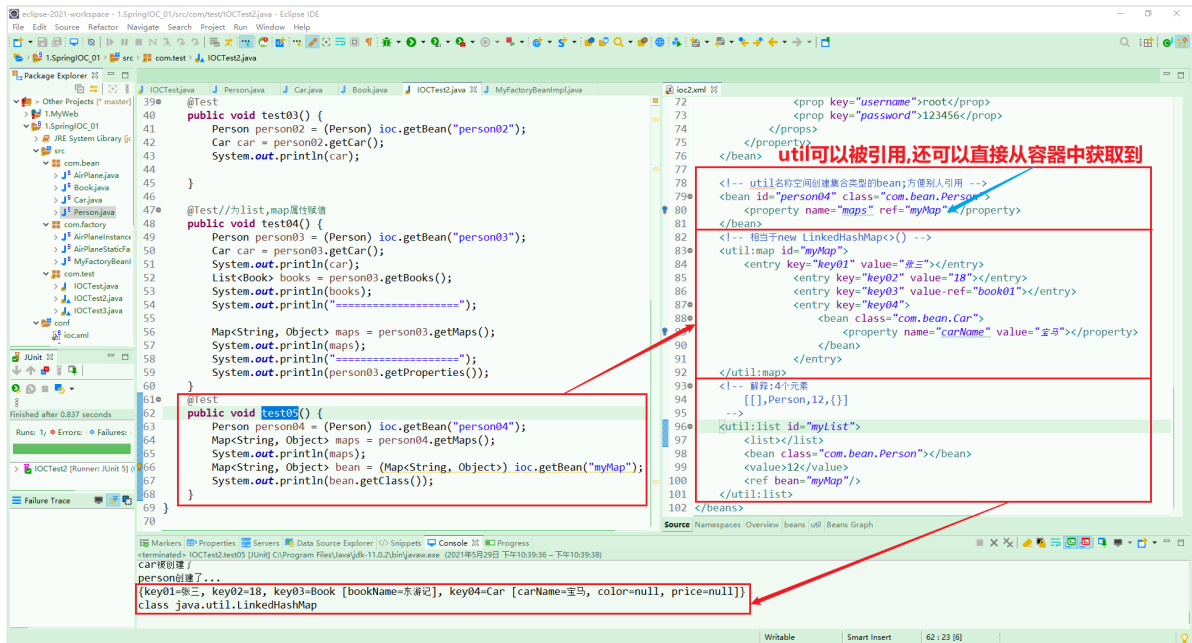
1.6.5. 为map属性赋值(雷神25集)



1.6.6. 为properties属性赋值(雷神26集)



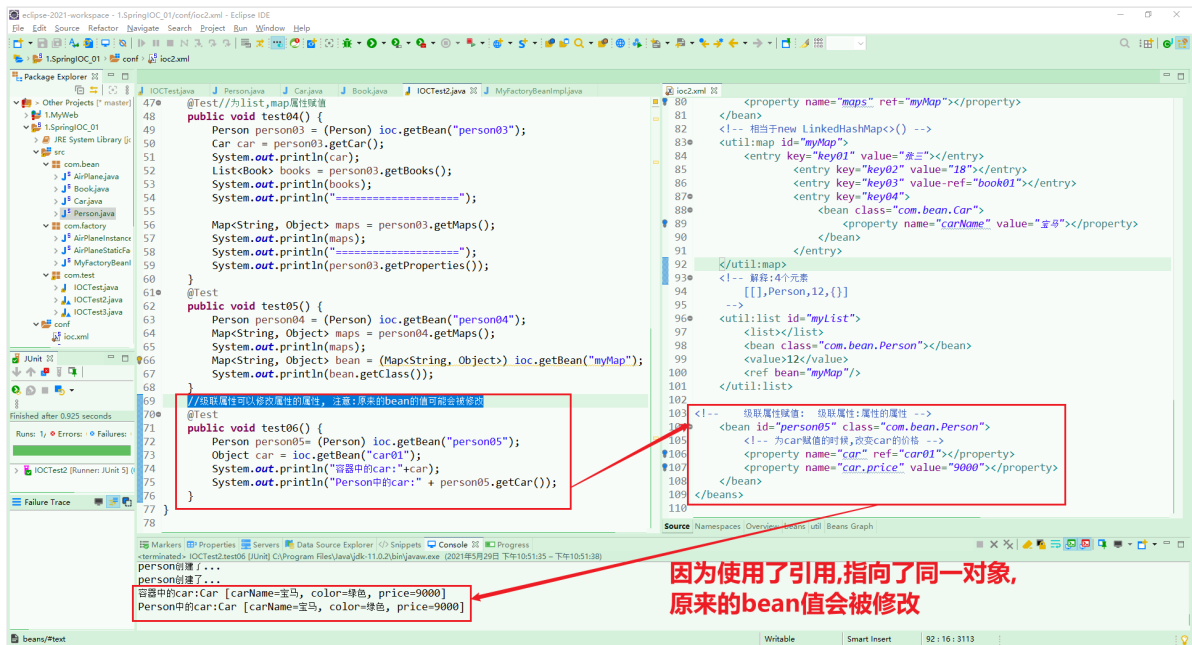
1.6.7. util名称空间创建集合类型的bean, 方便别人引用



1.6.8. 级联属性赋值

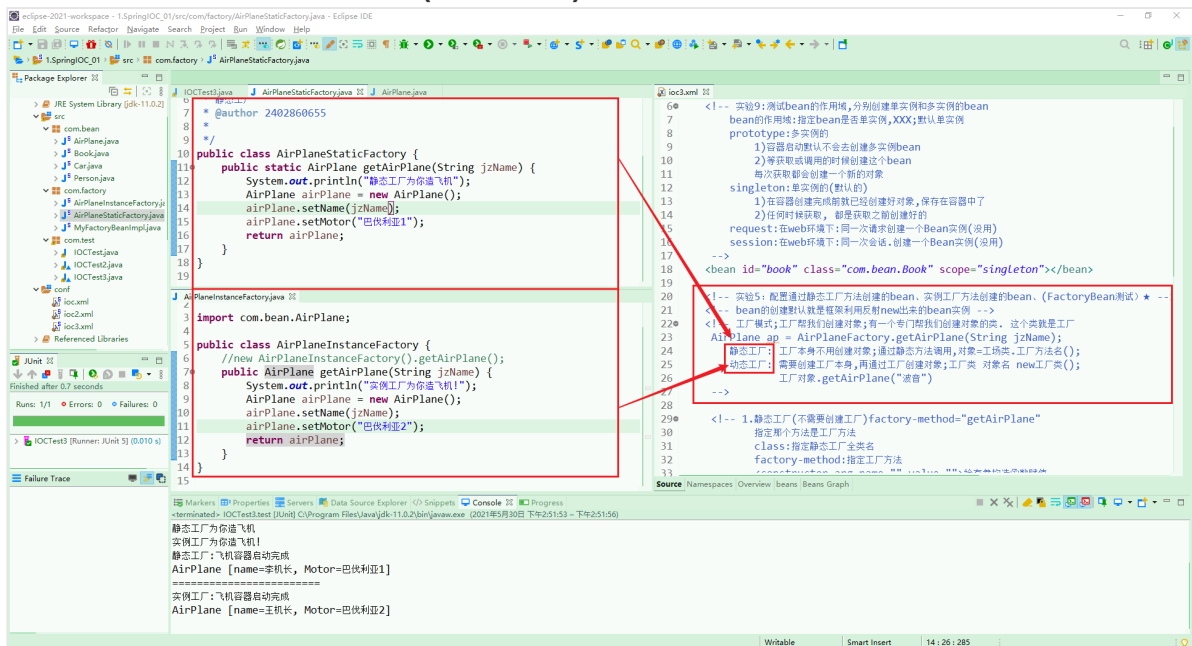
级联属性:属性的属性

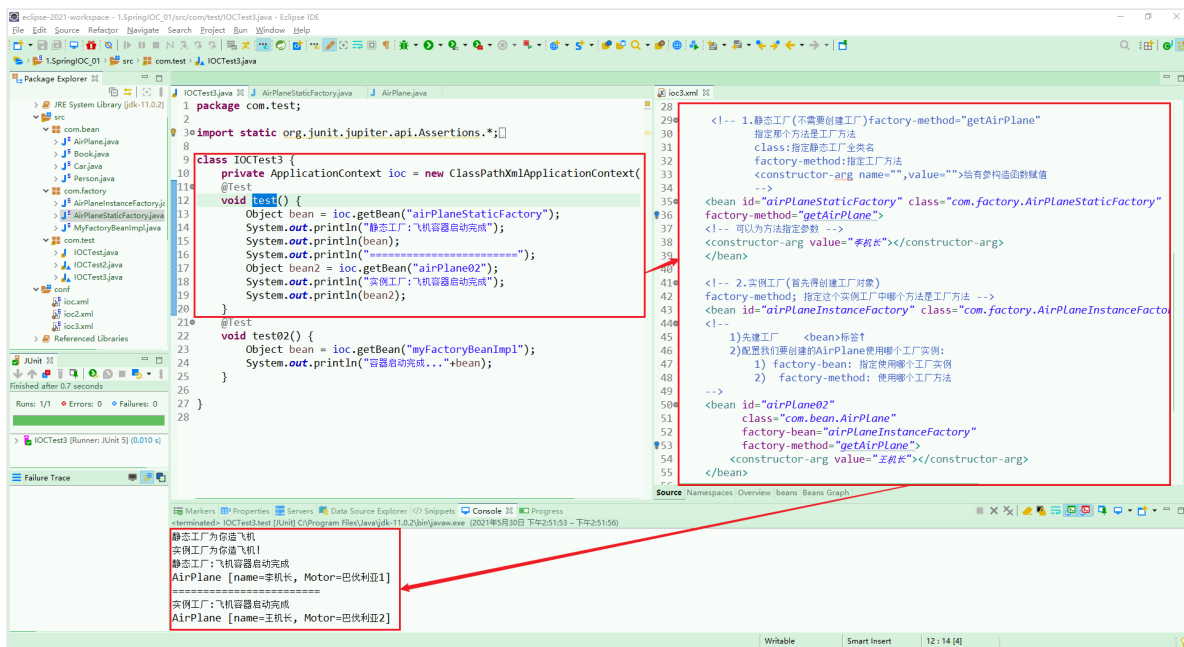
级联属性可以修改属性的属性, 注意:原来的bean的值可能会被修改



1.7. 实验5：配置通过静态工厂方法创建的bean、实例工厂方法创建的bean、FactoryBean★(雷神37集)

1.7.1. 静态工厂与动态工厂(实例工厂):





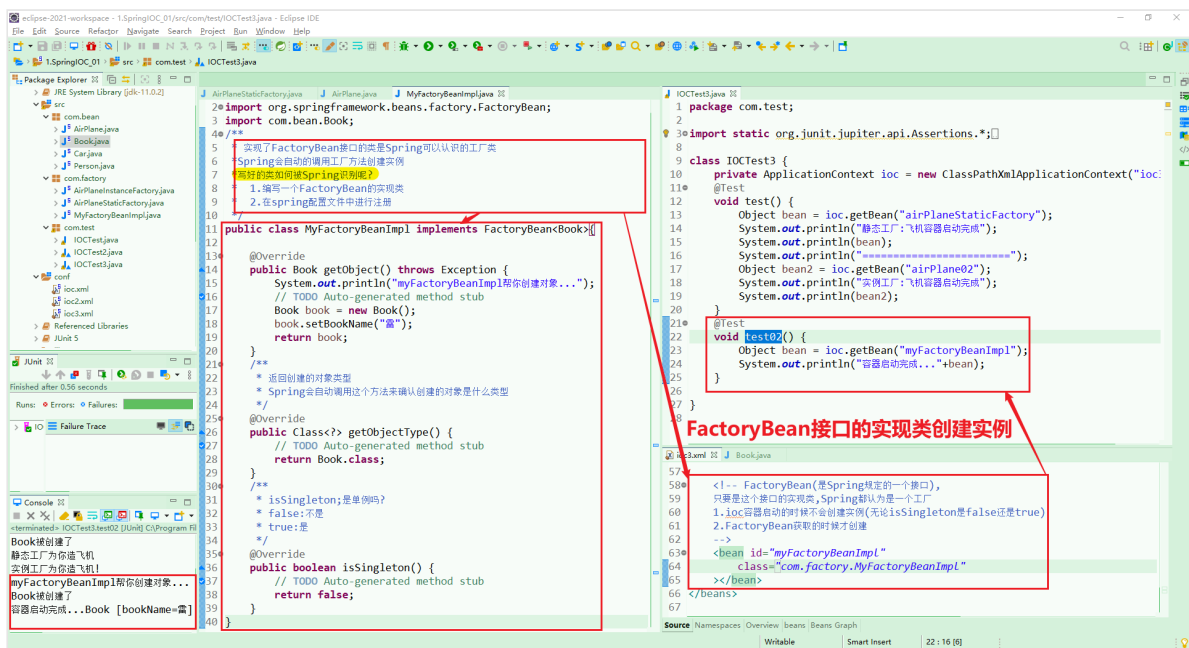
1.7.2. FactoryBean★(是Spring规定的一个接口)

只要是实现这个接口的类, Spring都认为是一个工厂

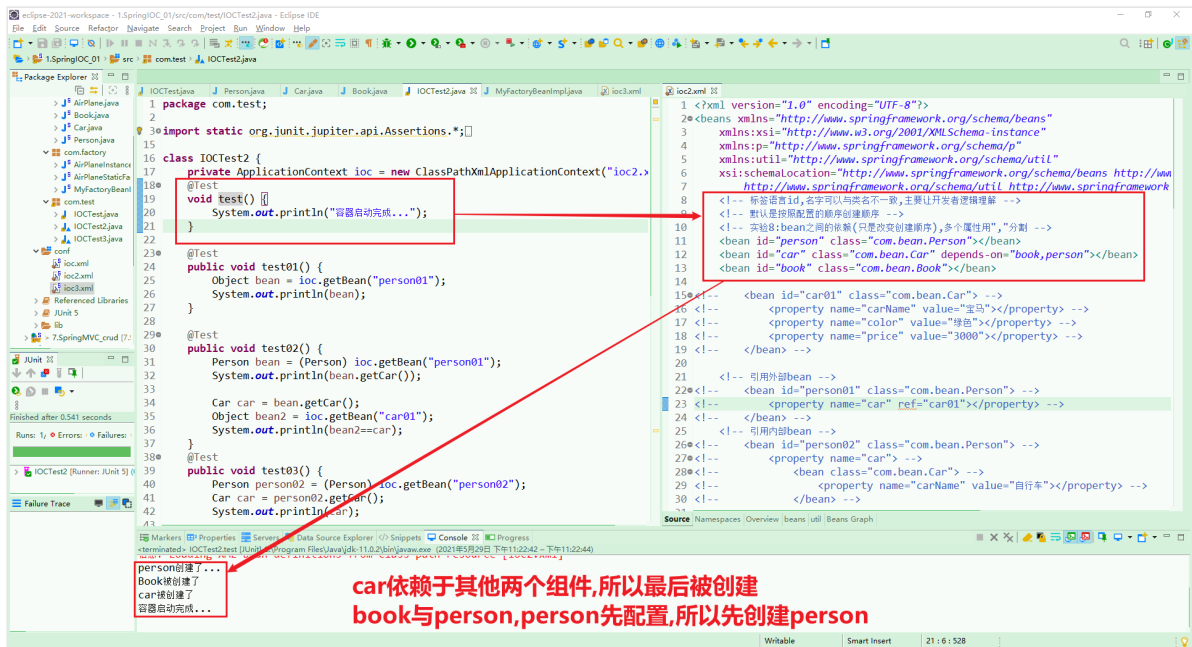
Spring会自动的调用工厂方法创建实例

写好的类如何被Spring识别呢?

- 1. 编写一个FactoryBean的实现类
- 2. 在spring配置文件中注册



1.8. 实验6:通过继承实现bean配置信息的重用(雷神29集)



1.11. 实验9: 测试bean的作用域, 分别创建单实例和多实例的bean★

bean的作用域:指定bean是否单实例,XXX;默认单实例

prototype:多实例的

- 1)容器启动默认不会去创建多实例bean
 - 2)等获取或调用的时候创建这个bean
- 每次获取都会创建一个新的对象

singleton:单实例的(默认的)

- 1)在容器创建完成前就已经创建好对象,保存在容器中了
- 2)任何时候获取, 都是获取之前创建好的

request:在web环境下:同一次请求创建一个Bean实例(没用)

session:在web环境下:同一次会话.创建一个Bean实例(没用)

<!-- 实验9:测试bean的作用域, 分别创建单实例和多实例的bean

bean的作用域:指定bean是否单实例,XXX;默认单实例

prototype:多实例的

- 1) 容器启动默认不会去创建多实例bean
 - 2) 等获取或调用的时候创建这个bean
- 每次获取都会创建一个新的对象

singleton:单实例的(默认的)

- 1)在容器创建完成前就已经创建好对象, 保存在容器中了
- 2)任何时候获取, 都是获取之前创建好的

request:在web环境下:同一次请求创建一个Bean实例(没用)

session:在web环境下:同一次会话.创建一个Bean实例(没用)

-->

<bean id="book" class="com.bean.Book" scope="singleton"></bean>

<!-- 实验5: 配置通过静态工厂方法创建的bean

<!-- bean的创建默认就是框架利用反射new出来

<!-- 工厂模式;工厂帮我们创建对象;有一个专门

AirPlane an = AirPlaneFactory.get

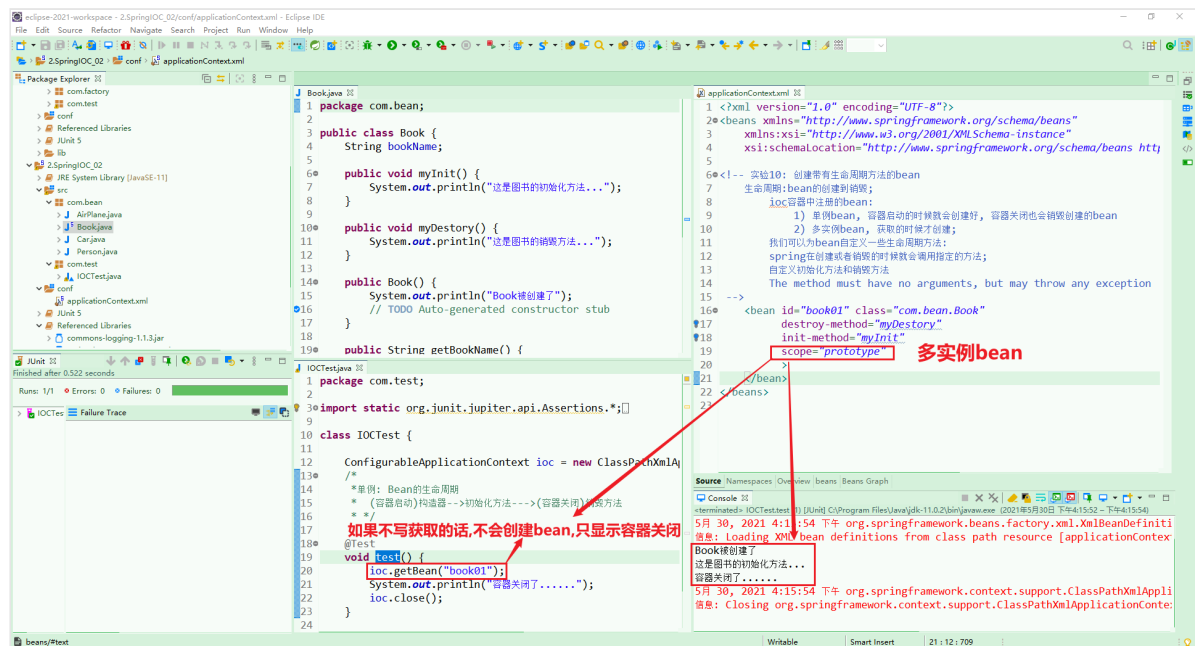
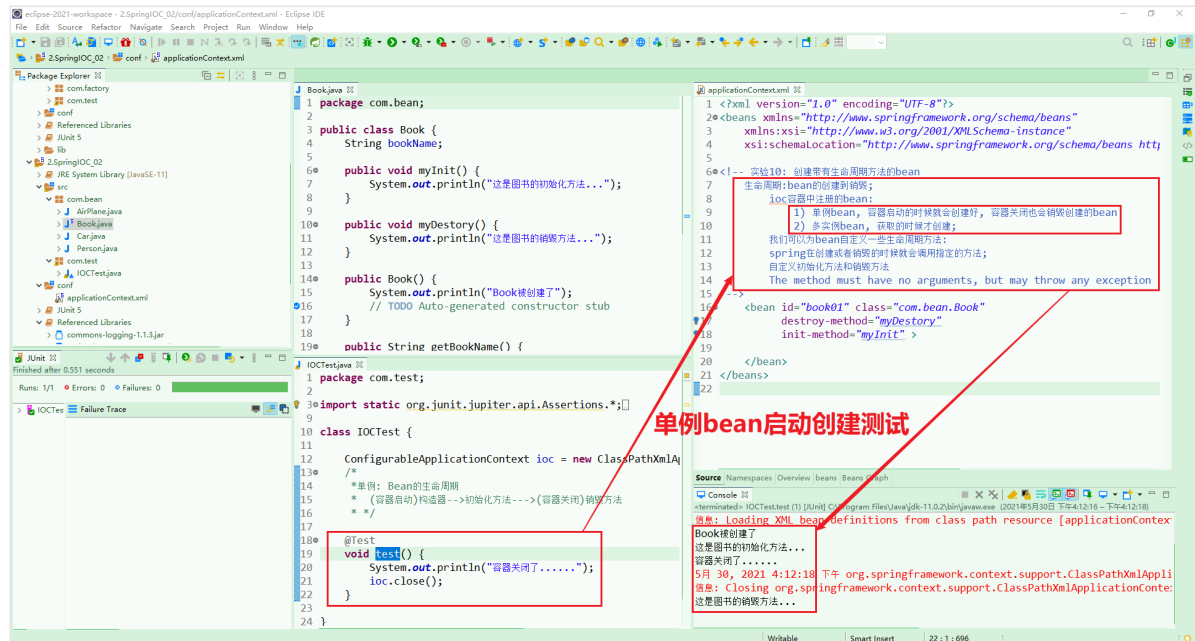
- prototype
- request
- session
- singleton

1.12. 实验10:创建带有生命周期方法的bean

Bean的生命周期

单例:(容器启动)构造器-->初始化方法--->(容器关闭)销毁方法

多实例:获取bean(构造器--->初始化方法)--->容器关闭不会调用bean的销毁方法

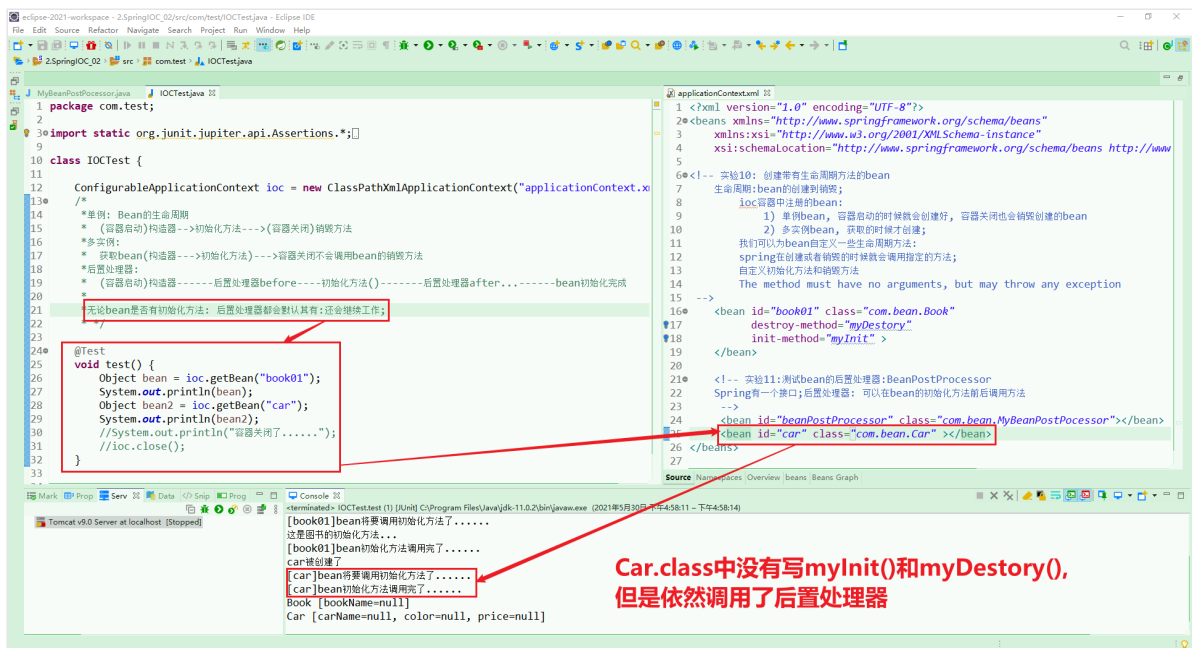
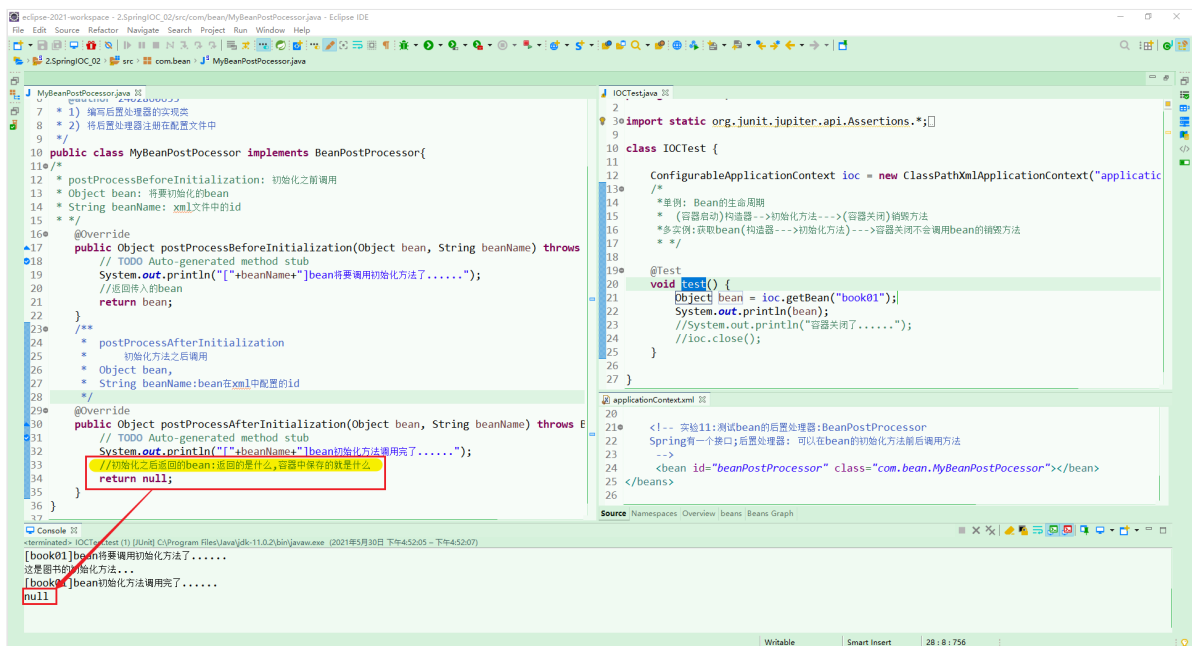
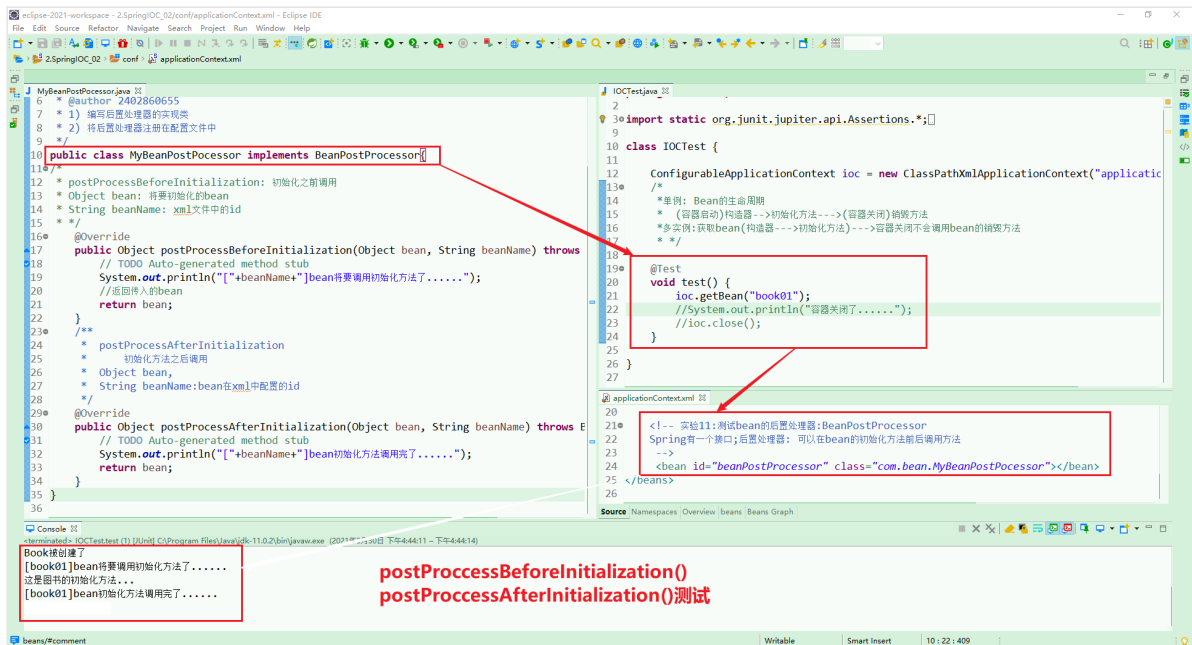


可以看到调用了bean的构造器生成的bean对象，并调用了初始化方法，但是就算关闭容器，也并不会调用销毁方法。容器对多实例的bean不进行管理。

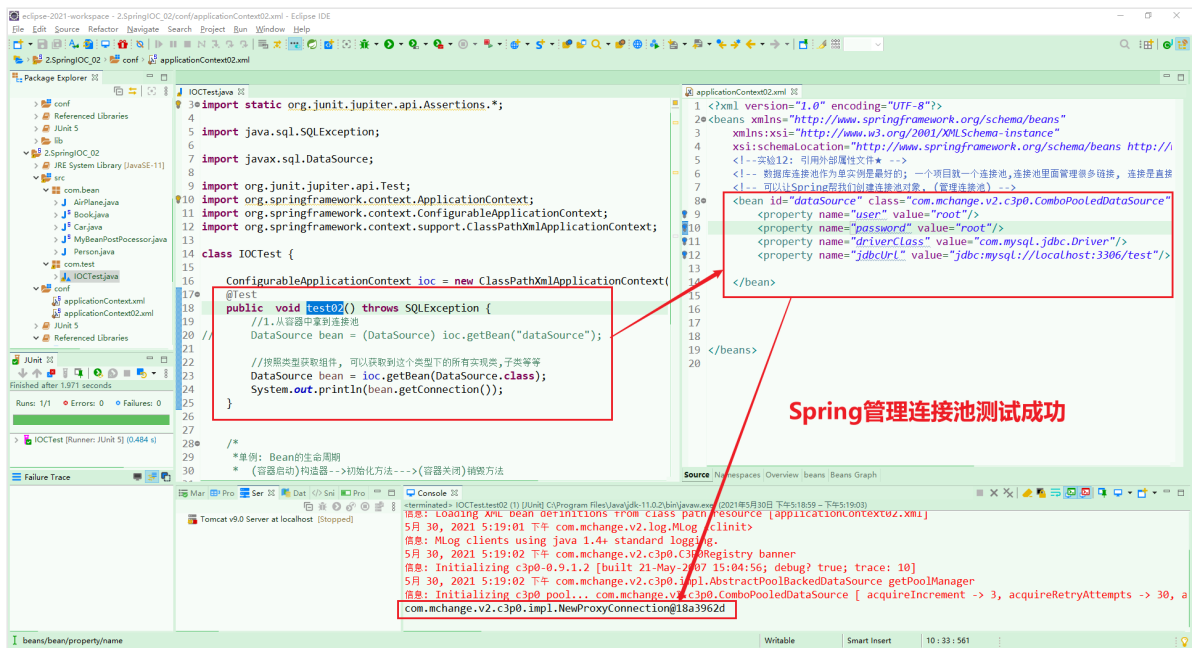
1.13. 实验11:测试bean的后置处理器(雷神39集)

后置处理器:

(容器启动)构造器-----后置处理器before-----初始化方法()-----后置处理器after...-----bean初始化完成

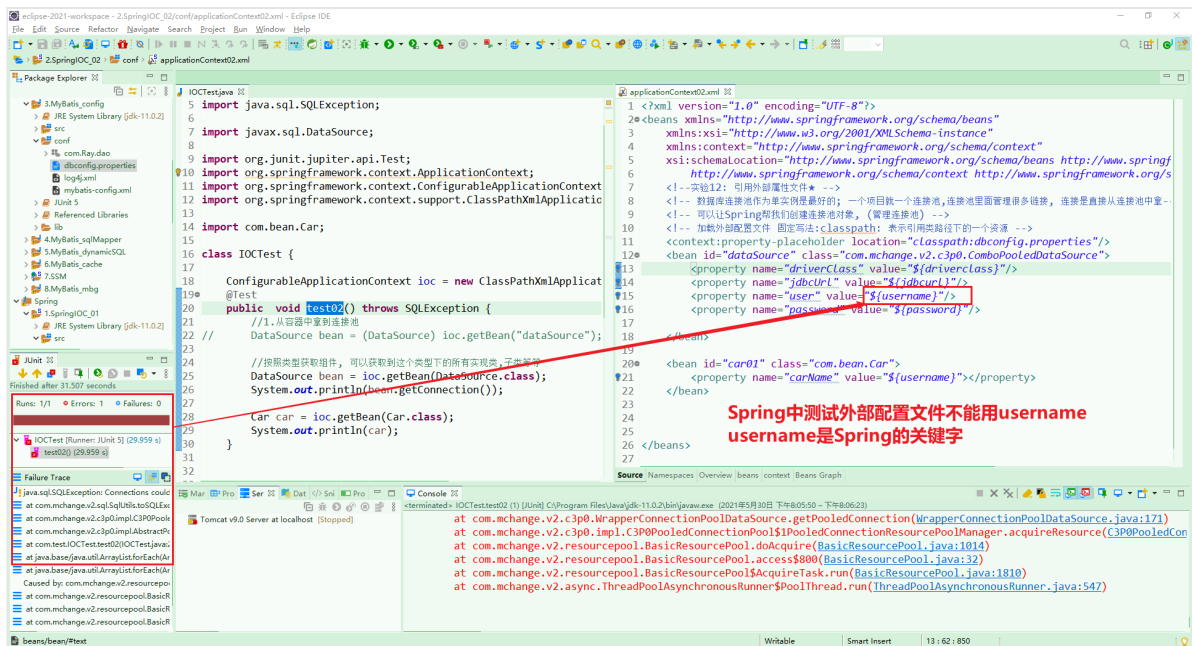


1.14. Spring管理连接池(雷神39集)



Spring管理连接池测试成功

1.15. 实验12:spring管理连接池引用外部配置文件★(雷神41集)

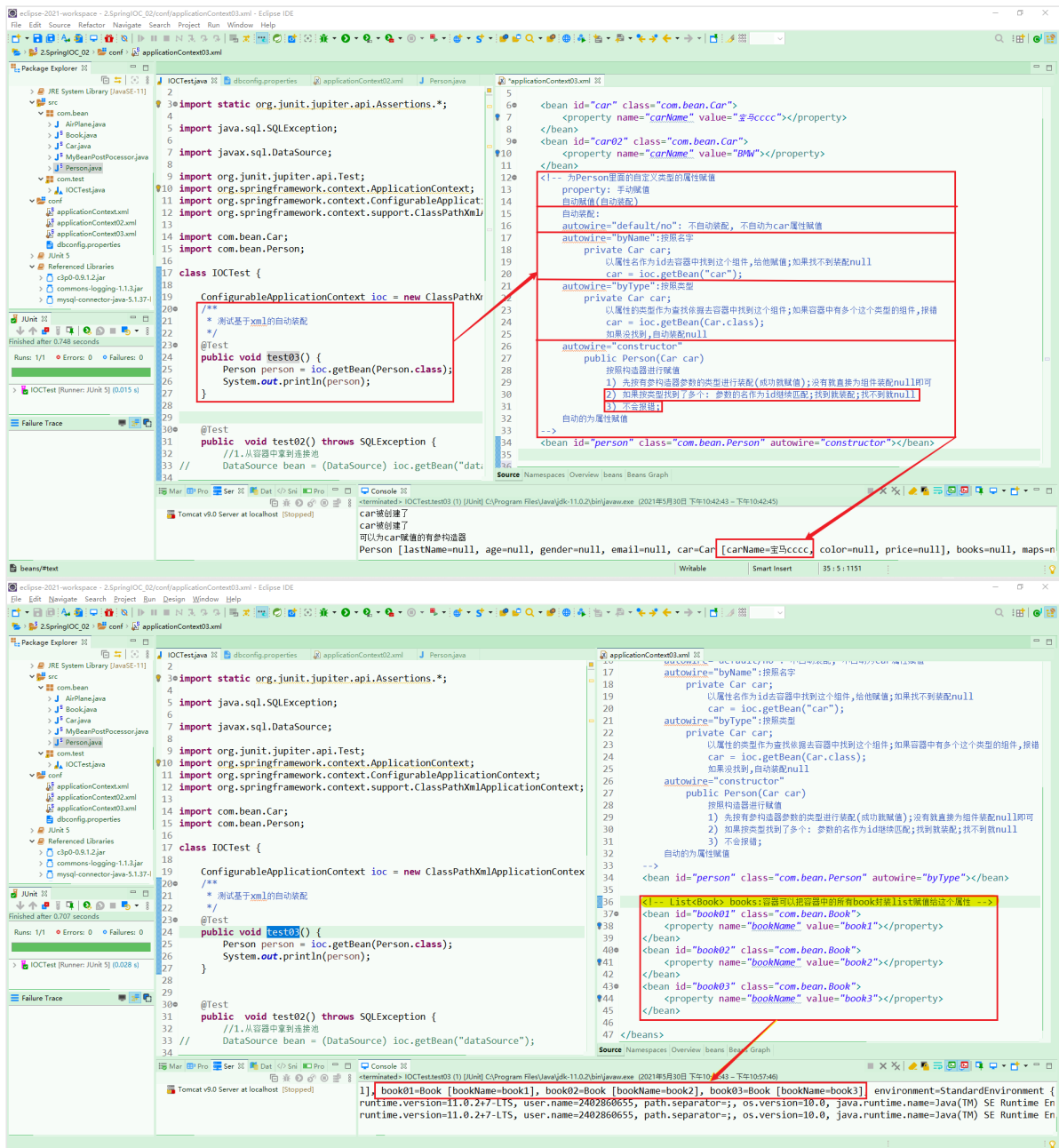


Spring中测试外部配置文件不能用username
username是Spring的关键字

起名推荐加前缀:jdbc

```
jdbc.driverclass=com.mysql.jdbc.Driver
jdbc.jdbcurl=jdbc:mysql://localhost:3306/mybatis_0325
jdbc.username=root
jdbc.password=root
```

1.16. 实验13:基于XML的自动装配(仅限对自定义类型自动赋值)



1.17. 实验14、SpEL测试(雷神44集)

- Spring 表达式语言，简称 SpEL，是一个支持运行时查询和操作对象图的强大表达式语言。
- 语法：使用 `#{...}` 作为定界符，为 bean 的属性进行动态赋值。

代码示例：

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://www.springframework.org/schema/beans
5         http://www.springframework.org/schema/beans/spring-beans.xsd">
6     <bean id="car" class="com.itdjx.spring.spel.Car">
7         <!--直接value="玛莎拉蒂"也可以-->
8         <property name="brand" value="#{'玛莎拉蒂'}"></property>
9         <!-- 或者
10        <property name="brand" value='#{"玛莎拉蒂"}'></property>
11        -->
12        <!--直接value="32000.78"也可以-->
13        <property name="price" value="#{32000.78}"></property>
  
```

```

14         <!--调用静态方法的属性 #{T(静态全类名).静态方法名(1,2)}-->
15         <property name="perimeter" value="#{T(java.lang.Math).PI *
75.8f}"></property>
16     </bean>
17
18     <bean id="person" class="com.itdjx.spring.spel.Person">
19         <!--调用对象的方法或属性，判断表达式#{true}/#{false}，marriage为
Boolean型-->
20         <property name="marriage" value="#{car.price > 400000 and age >
30}"></property>
21         <!--引用bean，之前为：<property name="car" ref="car"></property>--
>
22         <property name="car" value="#{car}"></property>
23         <property name="socialStatus" value="#{car.price > 30000 ? '金
领' : '白领'}"></property>
24         <property name="address" value="#{address.province + '省' +
address.city + '市' + address.area + '区'}"/>
25     </bean>
26
27     <bean id="address" class="com.itdjx.spring.spel.Address">
28         <property name="province" value="#{'辽宁'}"/>
29         <property name="city" value="#{'大连'}"/>
30         <property name="area" value="#{'沙河口'}"/>
31     </bean>
32
33 </beans>

```

- SpEL

- 整数: #{8}
- 小数: #{8.8}
- 科学计数法: #{1e4}
- String: 使用单或双引号作为字符串的定界符
- Boolean: #{true}
- 引用对象: #{car}
- 引用对象属性: #{car.brand}
- 调用对象方法: #{car.toString()}, 还可以链式操作
- 调用静态方法静态属性、静态方法: #{T(java.lang.Math).PI}, #{T(java.lang.Math).random()}
使用 T() 调用类作用域的方法和常量
- 支持算术运算符: +, -, *, /, %, ^ (加号还可以用作字符串连接)
- 比较运算符: /gt, ==/eq, >=/ge, <=/le
- 逻辑运算符: and, or, not, |
- 三目运算符: ? :
- 正则表达式: #{admin.email matches '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+.[a-zA-Z]{2,4}'}
- 避免抛出空指针异常: 加个?, 它会确保在左边项为 null 时不调用后面方法

```

1 <property name="song" value="#{songSelector.selectSong()?.toUpperCase()}"
/>

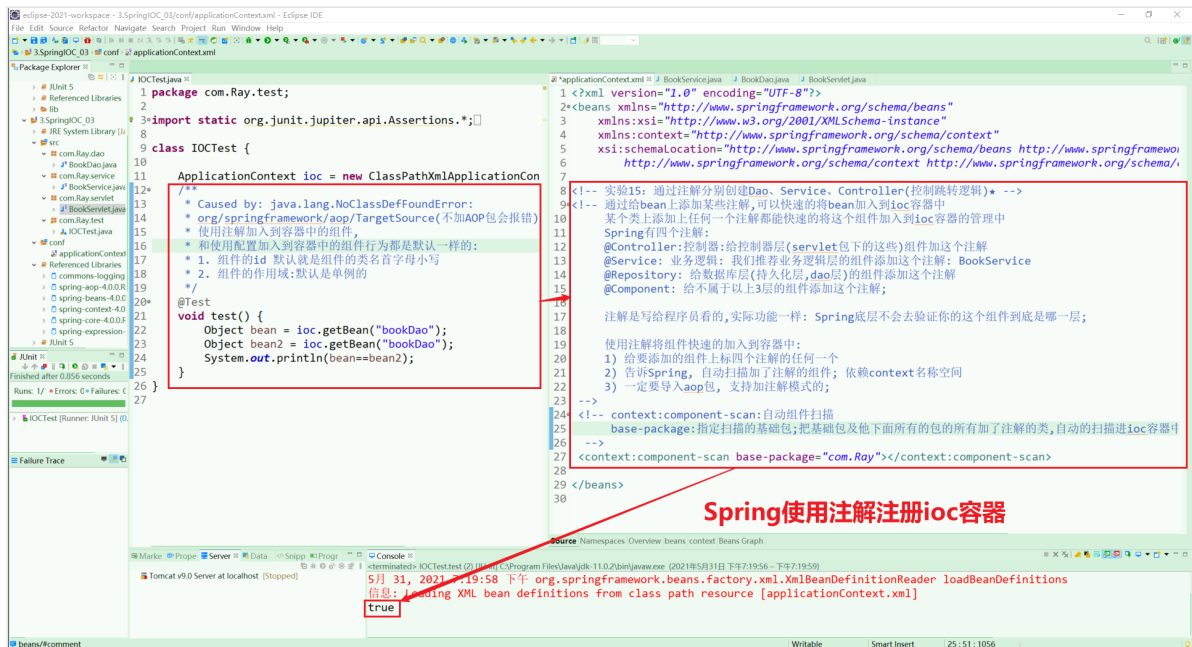
```

1.18. 实验15：通过注解分别创建Dao、Service、Controller★(雷神45集)

```
*applicationContext.xml  BookServlet.java
1 package com.Ray.servlet;
2
3 import org.springframework.stereotype.Controller;
4
5 @Controller
6 public class BookServlet {
7
8 }
9

BookDao.java
1 package com.Ray.dao;
2
3 import org.springframework.stereotype.Repository;
4
5 @Repository
6 public class BookDao {
7
8 }
9

BookService.java
1 package com.Ray.service;
2
3 import org.springframework.stereotype.Service;
4
5 @Service
6 public class BookService {
7
8 }
9
```

Spring使用注解注册ioc容器

```
/**
 * Caused by: java.lang.ClassNotFoundException:
 * org.springframework.aop.TargetSource(不加AOP包会报错,图中是没有BookDao.class报的错)
 * 使用注解加入到容器中的组件,
 * 和使用配置加入到容器中的组件行为都是默认一样的:
 * 1. 组件的id 默认就是组件的类名首字母小写
 * 2. 组件的作用域:默认是单例的
 */
```

<!-- 通过给bean上添加某些注解,可以快速的将bean加入到ioc容器中
某个类上添加上任何一个注解都能快速的将这个组件加入到ioc容器的管理中
Spring有四个注解:
@Controller:控制器:给控制器层(servlet包下的这些)组件加这个注解
@Service: 业务逻辑: 我们推荐业务逻辑层的组件添加这个注解: BookService
@Repository: 给数据库层(持久化层,dao层)的组件添加这个注解
@Component: 给不属于以上3层的组件添加这个注解;

- 1 注解是写给程序员看的,实际功能一样: Spring底层不会去验证你的这个组件到底是哪一层;
- 2
- 3 使用注解将组件快速的加入到容器中:
- 4 1) 给要添加的组件上标四个注解的任何一个
- 5 2) 告诉Spring, 自动扫描加了注解的组件; 依赖context名称空间
- 6 3) 一定要导入aop包, 支持加注解模式的;

-->

基于注解加入到ioc容器中组件的默认行为调整(雷神46集)

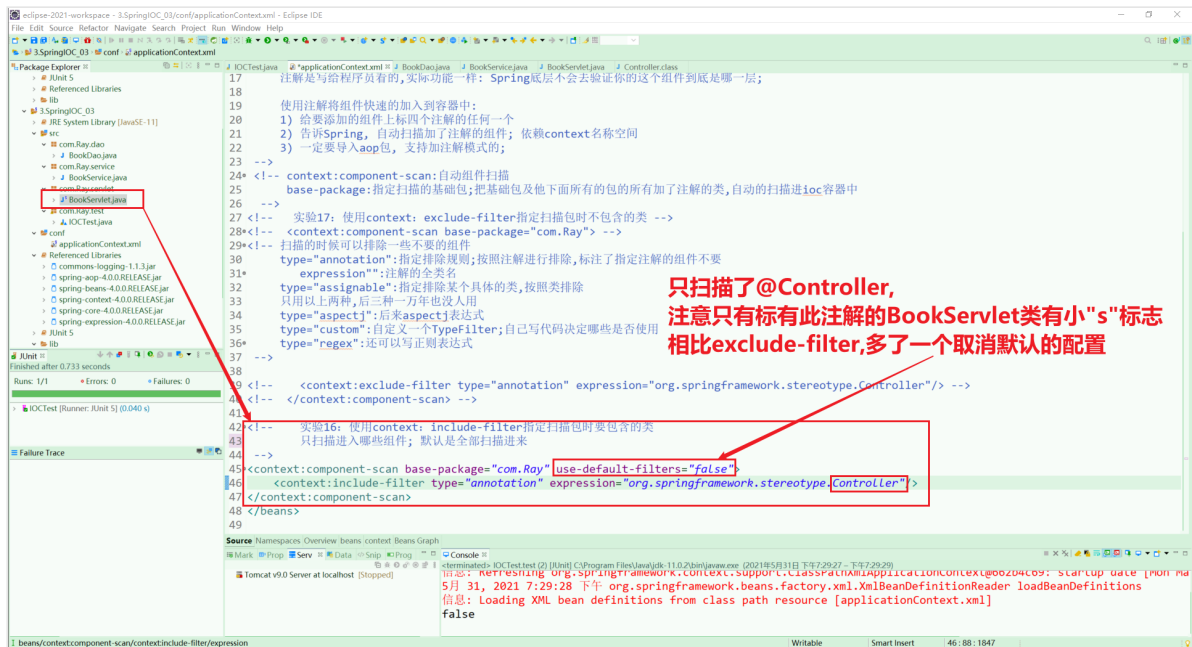


注意:注解不能把别人的jar包加入注解, 但使用XML可以, 注解与XML相结合使用

1.19. 实验17: 使用context: exclude-filter指定扫描包时不包含的类(雷神47集)

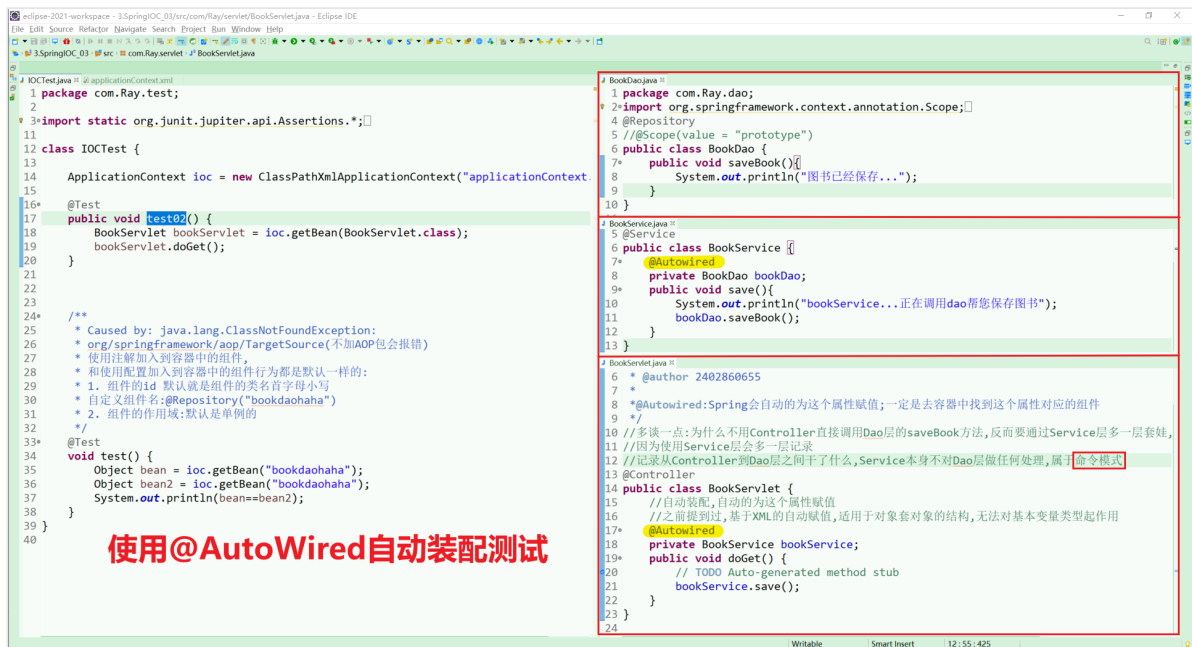


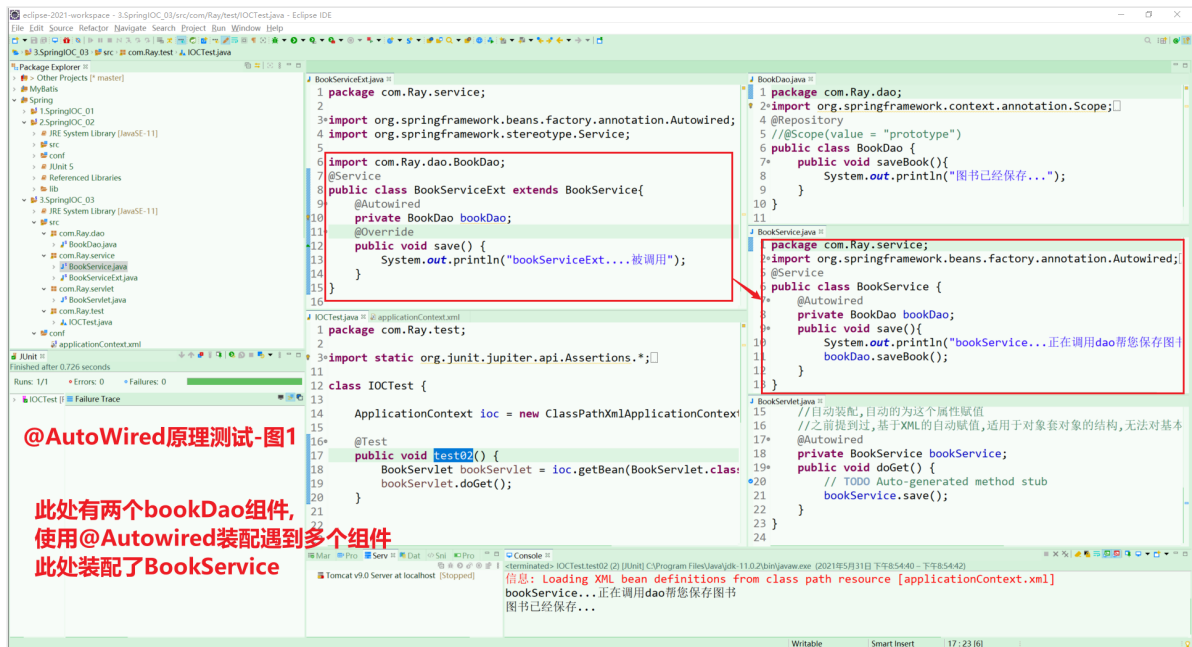
1.20. 实验16: 使用context: include-filter指定扫描包时要包含的类(雷神48集)



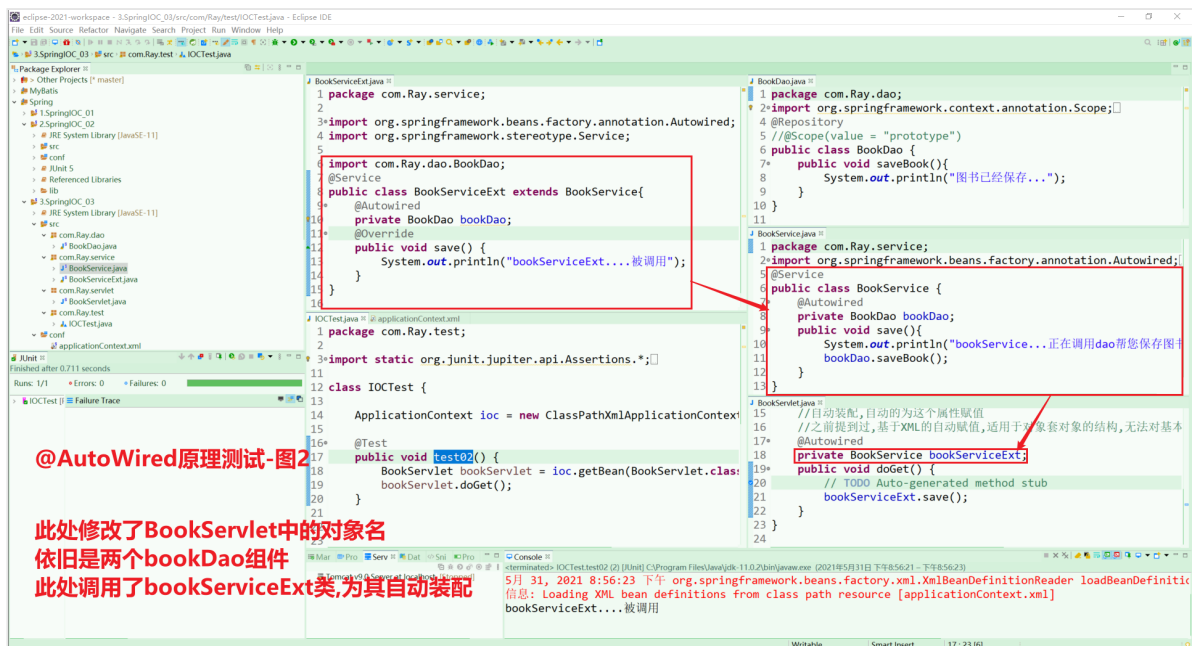
```
<context:component-scan base-package="com.Ray" use-default-filters="false">
    <context:include-filter type="annotation"
        expression="org.springframework.stereotype.Controller"/>
</context:component-scan>
```

1.21. 实验18：使用@Autowired注解实现根据类型实现自动装配★(雷神49集)





1.22. 实验19：如果资源类型的bean不止一个，默认根据@Autowired注解标记的成员变量名作为id查找bean，进行装配★

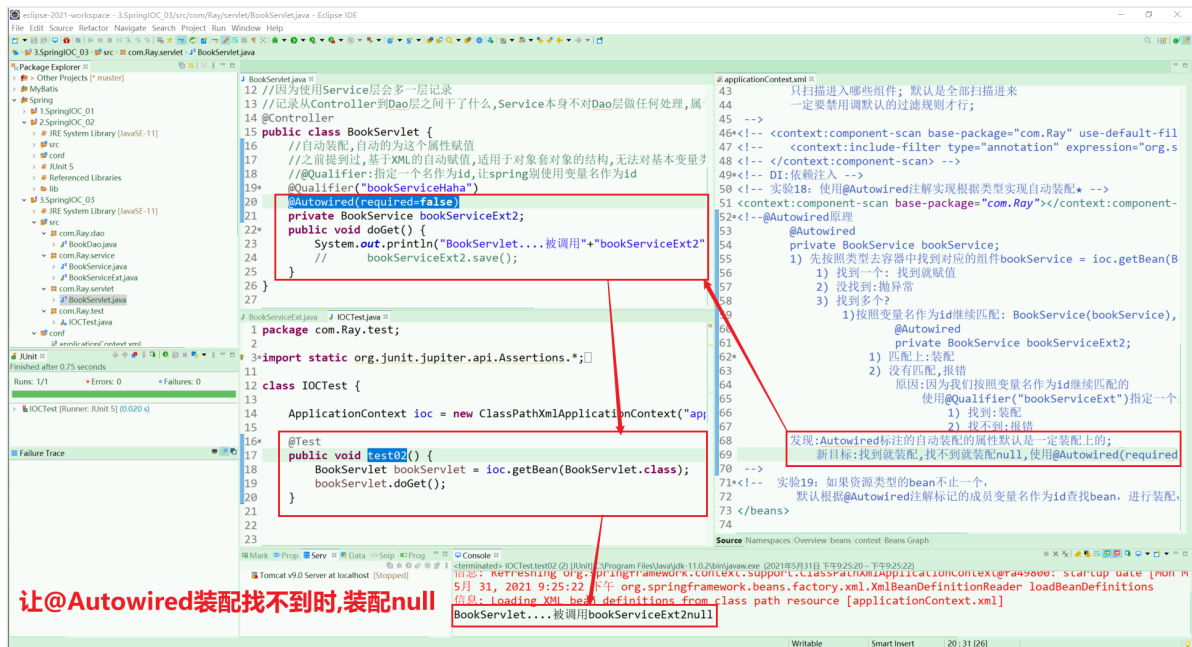


1.23. 实验20：如果根据成员变量名作为id还是找不到bean，可以使用@Qualifier注解明确指定目标bean的id★



注意:此处应该标bookServiceExt来为bookServiceExt对象装配, 但是使用了@Qualifier("bookService")后,给bookService对象装配了

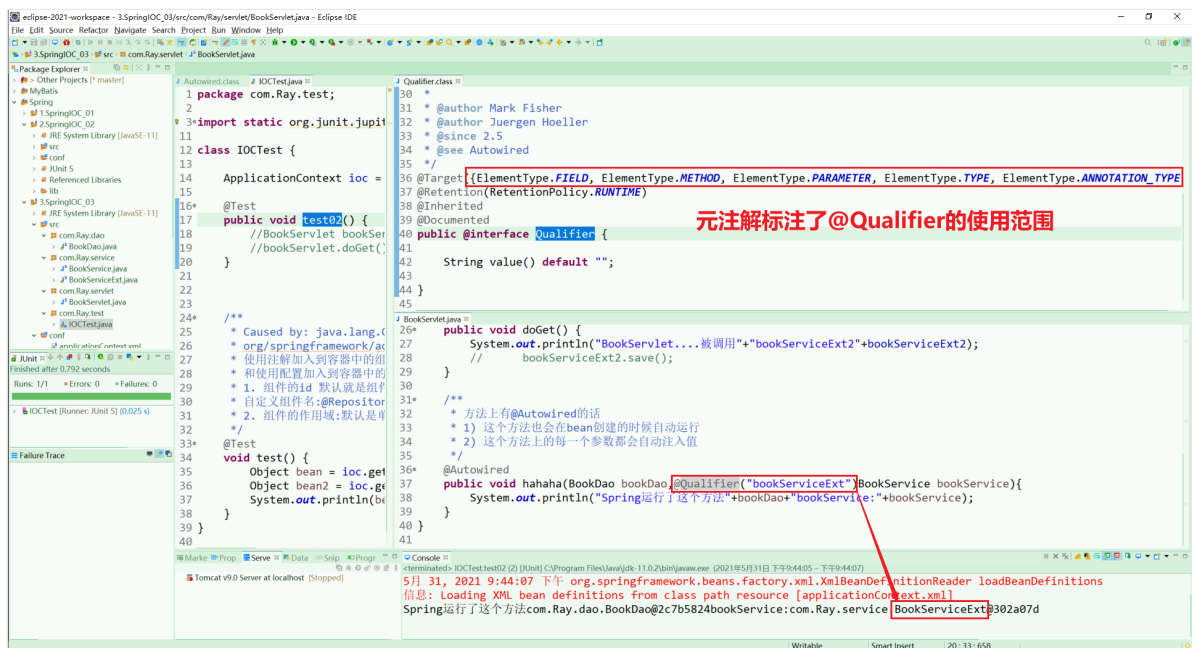
1.24. 实验22: @Autowired注解的required属性指定某个属性允许不被设置(装配为null)(雷神52集)



1.25. @Autowired流程总结(雷神53集)

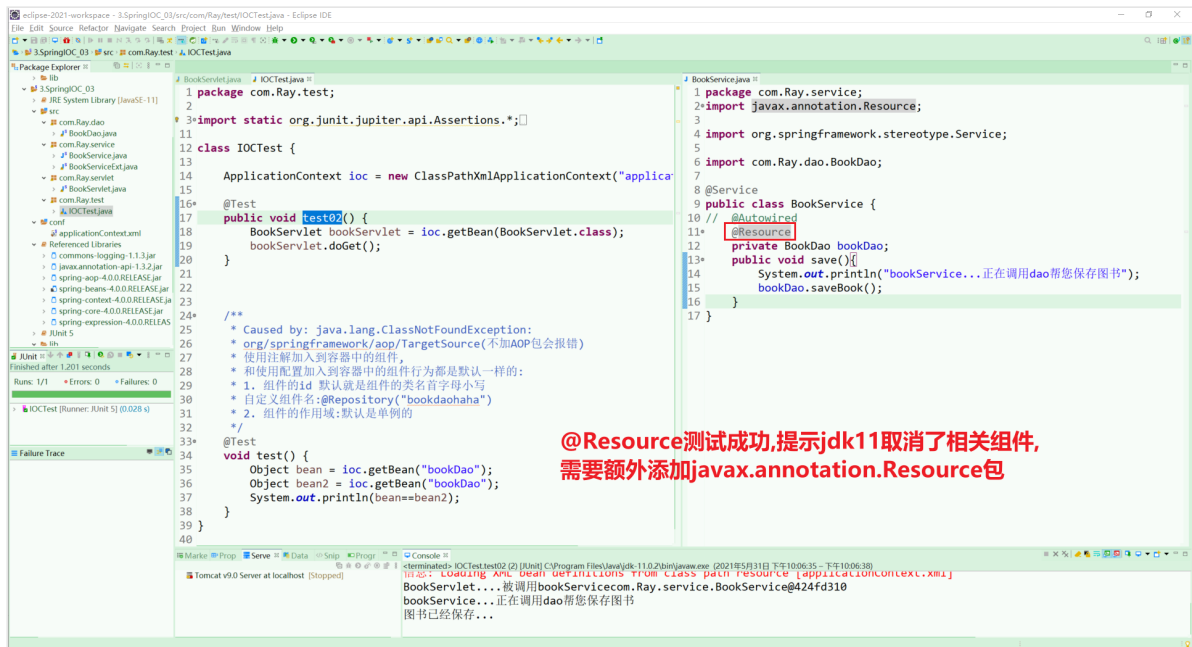
```
1 <!--@Autowired原理
2 @Autowired
3 private BookService bookService;
4 1) 先按照类型去容器中找到对应的组件bookService =
ioc.getBean(BookService.class);
5 1) 找到一个: 找到就赋值
6 2) 没找到:抛异常
7 3) 找到多个?
8 1)按照变量名作为id继续匹配: BookService(bookService),
BookServiceExt(bookServiceExt)
9 @Autowired
10 private BookService bookServiceExt2;
11 1) 匹配上:装配
12 2) 没有匹配,报错
13 原因:因为我们按照变量名作为id继续匹配的
14 使用@Qualifier("bookServiceExt")指定一个新id
15 1) 找到:装配
16 2) 找不到:报错
17 发现:Autowired标注的自动装配的属性默认是一定装配上的;
18 新目标:找到就装配,找不到就装配null,使用@Autowired(required=false)
19 -->
```

1.26. 实验21: 在方法的形参位置使用@Qualifier注解(雷神54集)



1.27. @AutoWired和@Resource的区别(雷神55集)

提示:jdk11去除了@Resource相关的包,需要引入

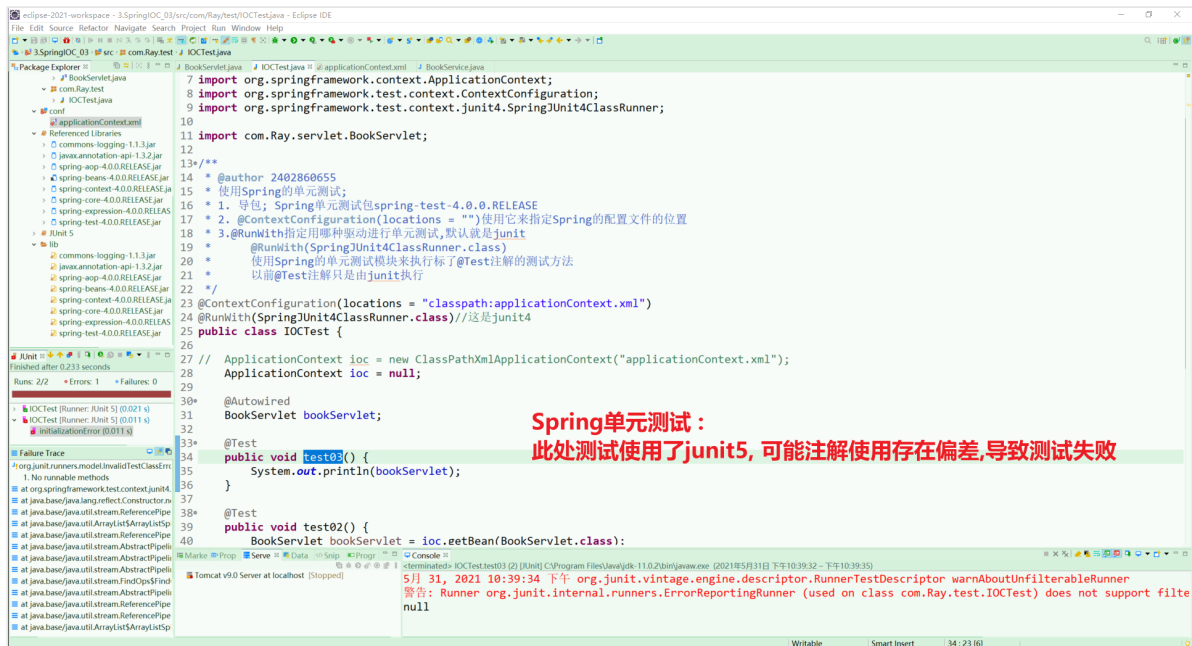


@Resource测试成功,提示jdk11取消了相关组件,需要额外添加javax.annotation.Resource包

- 1 <!-- @Autowired @Resource @Inject都是自动装配的意思 -->
- 2 @Autowired:最强大;Spring自己的注解
- 3 @Resource: j2ee:java的标准,扩展性更强;如果切换成另外一个容器框架(solon):@Resource还能用
- 4 @Inject: EJB开发(不用管)

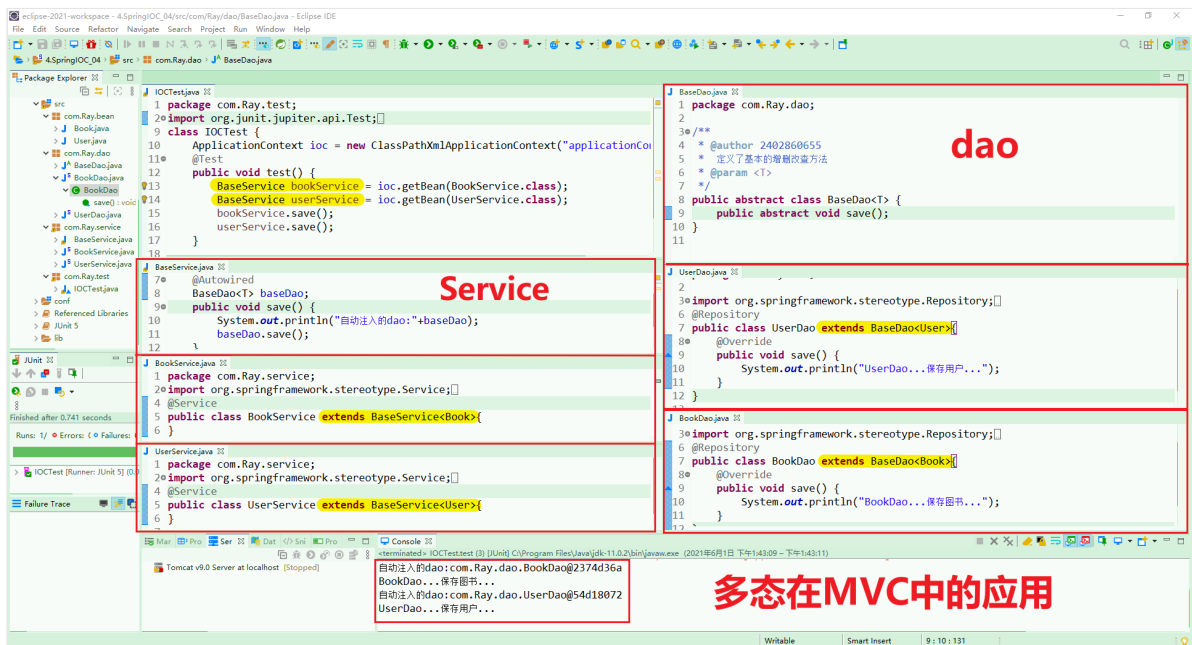
1.28. Spring的单元测试

spring-test-4.0.0.RELEASE

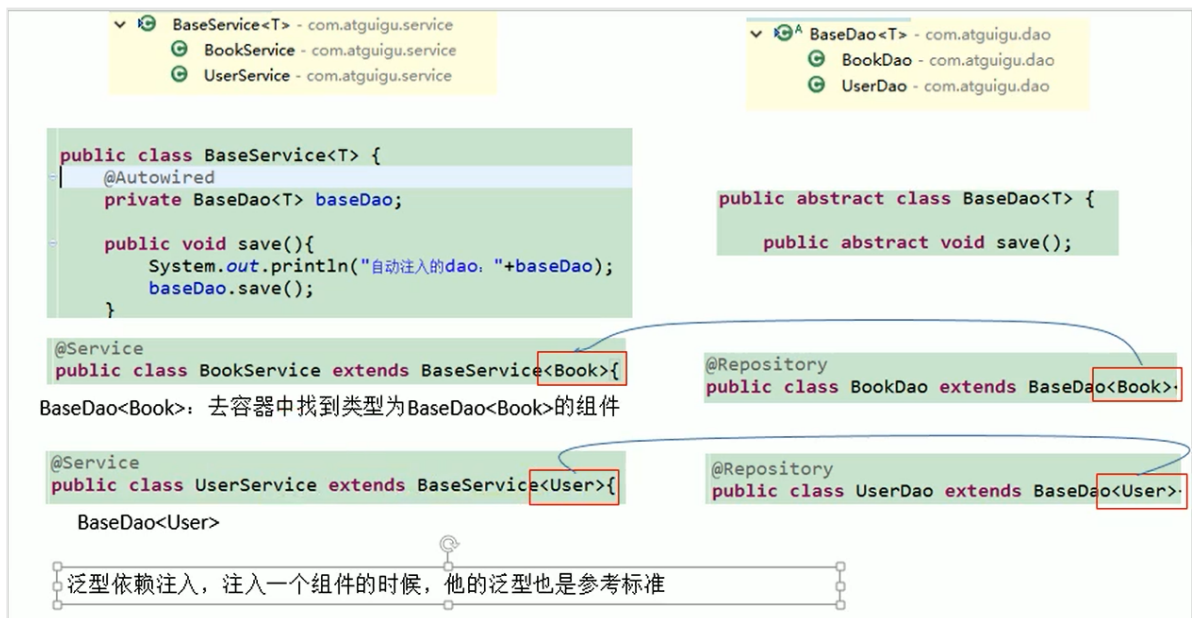


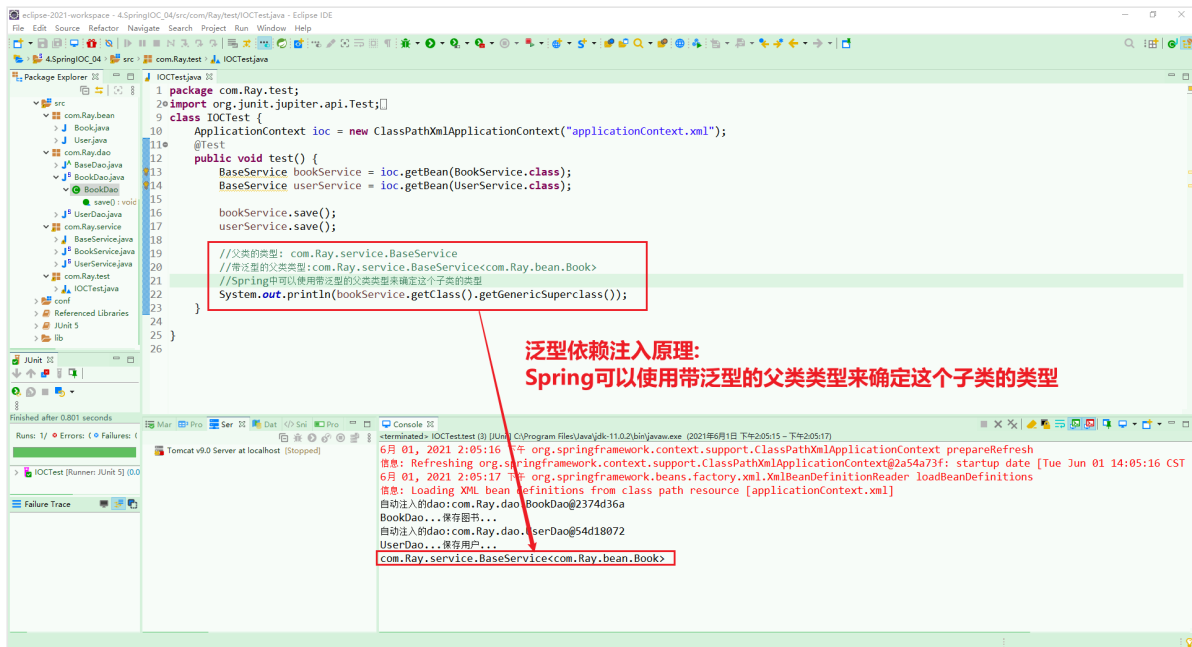
Spring单元测试：
此处测试使用了junit5, 可能注解使用存在偏差,导致测试失败

1.29. 实验23：测试泛型依赖注入★



1.30. 泛型依赖注入原理★





1.31. ioc总结:(雷神60集)

ioc是一个容器,帮我们管理所有的组件

1. 依赖注入: @Autowired; 自动赋值(哪些组件需要用另一些组件,一个Autowired自动搞定)
2. 某个组件要使用Spring提供的更多功能(IOC,AOP)必须加入到容器中

体会:

1. 容器启动 创建所有单实例bean
2. autowired自动装配的时候,是从容器中找这些符合要求的bean
3. ioc.getBean("bookServlet"); 也是从容器中找到的这个bean;
4. 容器中包括了所有的bean
5. 调试Spring的源码, 容器到底是什么?其实就是一个map
6. 这个map中保存了所有创建好的bean,并提供外界获取功能...
7. 探索: 单实例的bean都保存到哪个map中了
8. 源码调试思路:

从HelloWorld开始的: 给helloworld每一个关键步骤打上断点; 进去看里面都做了什么工作

怎么知道哪些方法都是干什么的?

1. 翻译这个方法
2. 放行这个方法, 看控制台, 看debug的每一个变量的变化
3. 看方法注释

学到的一点: 1)规范注释 2)规范方法名和类名